Cartera: el pool, el universo, las parejas y la búsqueda de fondeo

AlgoProject — manual de uso. Actualizado el 2026-10-05 con `tools/manual.py`.

1. 77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo archivarlas
2. 78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas
3. 79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan
4. 87. Cartera · la etapa B y el veredicto de fondeo — ¿aguanta la cartera elegida un Monte Carlo, y fuera de muestra?
5. 88. La caja del fondeo — lo que sale de tu banco contra lo que vuelve, plan por plan
6. 96. Cartera: la búsqueda por rangos — qué estrategias del pool van juntas, elegidas en build
7. 100. Cartera · la raíz de prueba — probar el motor de carteras sin tocar los datos de verdad
8. 101. Cartera · el veredicto de la cuenta real — ¿le gana la cartera elegida al azar, y es real su Sharpe?
9. 102. La decisión del fondeo — qué cuenta comprar, en una frase, con el coste al lado
10. 103. Cartera · el walk-forward de pesos — ¿algún método de pesos le gana a pesos iguales fuera de muestra?
11. 104. Cartera · la incubación — el registro de la cartera elegida y la lectura de sus días nuevos

77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo

archivarlas

Qué pregunta responde

¿Qué estrategias de tus proyectos pueden entrar en el pool con el que se desarrolla el motor de carteras, y con qué orden exacto se archiva cada una? El comando lee los veredictos que ya dejaron los estudios de cada paso y te imprime, estrategia por estrategia, la línea de `core.archive` que la congela. **No archiva nada**: tú apruebas la lista y lanzas las líneas que quieras.

Es el hito M0 del plan de carteras (`portfolio/PLAN.md`). Las carteras de este pool **no son operables**: son para desarrollar y probar el motor hasta que exista el pool validado del paso 26.

Cuándo lo usas, y cuándo no

Úsalo cuando un run del workflow ha terminado y quieres llevar sus estrategias al archivo para construir carteras con ellas. **No** lo uses mientras SQX está corriendo ese proyecto: la lista sale igual, pero el archivo se niega a congelar una estrategia de un proyecto que SQX tiene en marcha.

Antes de empezar

Nada. Solo lee `AlgoData/reports/` y `AlgoData/strategyPermutations/`; no toca SQX ni el demonio.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.candidates --project Test_USDJPY_donchianUpperCrossUp_H1
```

flag	obligatorio	qué hace
`--project`	no	un solo proyecto; sin él, todos los que tienen informes

Tarda unos segundos. No toca SQX.

Qué produce

Nada en disco: solo imprime. Por proyecto, una línea por candidata con su clase, su paso, sus palabras y la orden de archivo, y al final cuántas quedaron fuera y por qué palabra.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Superviviente**: la palabra del último paso que la juzgó es MANTENER, survives, worth_it, predicts o PASS.
- **Dudosa** (casi-superviviente): DUDOSA o INCONCLUSIVE. Entra al pool, marcada.
- **Lo que el workflow llevó adelante manda**: toda madre con lote de variantes en el 16.5 entra como superviviente del paso 16.5, digan lo que digan las palabras de antes. Su línea muestra **todas** esas palabras (también los DESCARTAR) para que lo veas al aprobarla.
- El FAIL de **mcRetest** no excluye: informa, no filtra (así lo hizo el workflow).
- Una estrategia ya archivada no se repite.

Una estrategia archivada con paso < 26 es **de desarrollo** automáticamente; no hay que marcarla.

Un ejemplo completo

Salida real del 2026-09-30, con el run H1 aún en el paso 16.5 (recortada: la lista entera tiene 170 líneas de candidatas):

```
=== Test_USDJPY_donchianUpperCrossUp_H1 ===
  [SUPERVIVIENTE] paso 16.5 · llevada al 16.5 · [crossTF control_failed, crossTF unusable, crossmarket DESCARTAR, decay DUDOSA, edgeCost MANTENER, feedQuality MANTENER, gate MANTENER, snoopingScreen MANTENER, spread MANTENER] · 032522d4cede…
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank Results --identity 032522d4cedef71ce06e3dbeaff3461a939f965ebb2530ea2999c88fcfbb782b --step 16.5 --family donchianUpperCrossUp
  [SUPERVIVIENTE] paso 8 · edgeCost · MANTENER · 003863ecc1d8…
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank Results --identity 003863ecc1d85d6b2f7b71d91be63f495c736fd80bbac5689693dfd4a4cbef45 --step 8 --family donchianUpperCrossUp
  …
  170 supervivientes, 0 dudosas, 0 ya archivadas
  descartadas por «DESCARTAR»: 114
  descartadas por «control_failed»: 3
  descartadas por «fails»: 1
  descartadas por «inherited»: 11
  descartadas por «unusable»: 1
```

Las 14 primeras líneas son las madres que llegaron al 16.5; las otras 156 son supervivientes del paso 8 que el workflow no llevó más lejos.

Después, para formar el pool se declara con un nombre (un pool declarado queda **congelado**: otro pool es otro nombre):

```
python3 -m portfolio.common.construct.inputs.pool declare Mi_pool --added-by Sergio --identity 032522d4cedef71ce06e3dbeaff3461a939f965ebb2530ea2999c88fcfbb782b --identity 003863ecc1d85d6b2f7b71d91be63f495c736fd80bbac5689693dfd4a4cbef45
```

flag	obligatorio	qué hace
`declare <nombre>`	sí	el nombre del pool; si ya existe, se niega y no toca nada
`--added-by`	sí	quién lo declara; queda escrito en cada fila
`--identity`	uno de los dos	una estrategia archivada, repetible; `identidad@versión` fija la versión, sin ella toma la última archivada
`--file`	uno de los dos	un archivo con una `identidad[@versión]` por línea
`--note`	no	una nota que se copia en cada fila

Escribe `AlgoData/portfolio/pools/<nombre>.csv` y contesta con el hash del pool. Salida real del 2026-10-05 (el pool de prueba, en la raíz de prueba del capítulo 100):

```
Pool 'Test_nikkei_ref' declarado: 10 miembros, hash ca050ebe3686… →
…/portfolio/_test/root/portfolio/pools/Test_nikkei_ref.csv
```

Declarar otra vez el mismo nombre contesta `el pool 'Test_nikkei_ref' ya está declarado: …` y no escribe. La ventana lo tendrá en la zona PORTFOLIOS.

Qué NO te dice

- **No te dice si una estrategia es buena**: solo que el workflow no la descartó hasta ese paso.
- **No sabe qué es un oos2 disponible**: el archivo guarda hoy build y oos1. Las madres del 16.5 tienen su curva de oos2 en el lote, pero sus operaciones de oos2 aún no existen en ningún export (2026-09-30) — por eso el veredicto de una cartera de desarrollo esperará a que el run termine.
- La etiqueta «paso 8 · edgeCost» nombra el primer estudio del paso: todos los del paso 8 dijeron MANTENER para esa estrategia.

Si algo falla

- `KeyError: 'strategy'` o `'identity'`: un `verdict.csv` antiguo sin esas columnas; se ignora el estudio que no tiene `identity`, pero uno con `identity` y sin `strategy` rompe: dilo.
- Una madre que esperabas no aparece: su lote en `strategyPermutations/<proyecto>/` aún no tiene carpeta `sqx/` (el 16.5 no ha llegado a ella).

78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas

Qué pregunta responde

Para un pool declarado: ¿cómo se movió la equity de cada estrategia **cada día y dentro del día** (flotante incluido, minuto a minuto desde M1), en el reloj de cada empresa de fondeo? ¿Cuadra con lo que dice el propio SQX? Y, con eso: ¿qué parejas de estrategias se parecen demasiado para ir en la misma cartera, y por qué filtro cae cada una?

Es el universo (hito M1) y el cribado de parejas (hito M2) del motor de carteras. No elige cartera todavía: eso es la búsqueda (F2 para fondeo), que espera a este paso.

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool declarado (capítulo 77) y quieras ver si sus estrategias se pueden combinar. **No** lo uses para juzgar una estrategia sola: el flotante y la reconciliación son solo el punto de partida.

Antes de empezar

- Un pool declarado en `AlgoData/portfolio/pools/<nombre>.csv`.
- Las barras M1 del feed de cada estrategia en `AlgoData/bars/` (USDJPY y XAUUSD están; los índices no, los importarás cuando haga falta).
- No toca SQX. Lee un fichero de SQX solo para saber el huso de cada feed, en modo lectura.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.universe --pool m1_test_uno
```

flag	obligatorio	qué hace
`--pool`	sí	el nombre del pool declarado
`--firm`	no	la empresa cuyas prohibiciones se aplican (`AlgoData/portfolio/prohibitions/<empresa>.csv`): las estrategias que esa empresa prohíbe salen del pool **antes** de buscar y se nombran una a una. Sin el flag no se aplica ninguna
`--set`	no	cambia un ajuste de `portfolio/common/construct/config.yaml`, p. ej. `--set pairs.pearson=0.45`. Un umbral distinto del tuyo marca el run como **AJUSTADO** en la primera línea

Tarda unos 7 s por estrategia (lee el M1 de cada feed una sola vez) y usa ~2,6 GB de RAM. La segunda vez con el mismo pool y la misma configuración lee la caché y tarda nada.

Qué produce

Una carpeta por pool, configuración **y versión del código que la construye** en `AlgoData/portfolio/universe/<hash del pool>-<hash de la config>-<hash del código>/` (una caché: se puede borrar, se rehace; si cambia el código del universo, la carpeta vieja deja de leerse y se construye otra):

fichero	qué es
`daily.parquet`	P&L diario a mercado de cada estrategia, en EET (tu reloj común), 0 dentro de su historia y vacío fuera
`monthly.parquet`	lo mismo por mes
`days_ftmo.parquet`, `days_hantec.parquet`	por día del servidor de cada empresa: lo cerrado, el flotante al cierre, el peor y el mejor momento del día
`m5_ftmo.npz`, `m5_hantec.npz`	el peor y el mejor flotante de cada bloque de 5 minutos, para sumar estrategias sin perder cuándo coinciden
`reconcile.csv`	cuánto cuadra cada estrategia con SQX
`pairs.parquet`	cada medida de cada pareja, solo sobre build
`manifest.json`	qué pool, qué config, qué relojes, qué versión del código

Cómo se lee el resultado

La reconciliación es la licencia para usar el flotante reconstruido. Dos pruebas contra SQX:

- **daily-low**: la curva diaria de SQX es, cada día, la equity **más baja** de ese día (lo cerrado más las posiciones abiertas en su peor mecha). El motor la reconstruye desde M1 y debe coincidir a menos de 1 $ en al menos el 99 % de los días. En la estrategia archivada coincide el 100 % (máx. 0,16 $).
- **MAE / MFE**: el peor y el mejor flotante de cada operación, contra las columnas `MAE ($)` y `MFE ($)` de SQX. En la archivada, 100 % y 99,9 %.

Una estrategia que no cuadra sale como **excluida**, con su motivo, y no entra en nada: nunca se usa en silencio.

Relojes sin confirmar: el del servidor de Hantec (Nueva York + 7 h) lo diste tú, sin confirmar; hasta leerlo del terminal MT5 de Hantec, todo lo de Hantec lo dice.

Las parejas (solo build, tus umbrales de 0,30):

- Pearson y Spearman, **mensual y diario**: cae si |ρ| > 0,30 en cualquiera.
- Co-loss (meses en que pierden las dos) y cola: caen si pasan de 0,30 **por arriba** (la cola entre dos estrategias independientes da −0,46 por cómo se eligen sus meses; por eso solo cuenta el lado positivo).
- Ventanas móviles de 60 meses (todo el build y las que acaban en sus últimos 36 meses): cae si el máximo |ρ| pasa de 0,30. Con menos de 60 meses compartidos no se aplican, y se cuenta.
- **Menos de 24 meses compartidos: fuera**, diga lo que diga el coeficiente.
- La correlación en días de estrés (el peor 5 % de días del pool) y el N efectivo **solo se informan**.

Un ejemplo completo

Salida real del 2026-09-30, pool de una sola estrategia (la archivada de USDJPY M30):

```
Universo del pool 'm1_test_uno' (c6a56a636ba5…)
Miembros: 1 conservados, 0 excluidos de 1
  4d679e0c2ce2… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
Calendario: build 2008-01-01…2017-12-31, oos1 2018-01-01…2022-12-31, oos2 2023-01-01…2026-08-30
Relojes sin confirmar: ['hantec']
Carpeta:
/home/sergioguslw/Desktop/AlgoData/portfolio/universe/c6a56a636ba5d09b44a7dd1bb9ffa74c149fc4846
463e6ca9ebfae09d2b5dc0b-07f85832e838bf0d
Tiempo: 6.81 s · RAM pico: 2631 MB
```

Con una sola estrategia no hay parejas. Con varias, debajo sale el cribado; así se lee en un pool de prueba de tres, dos de ellas casi clones:

```
Parejas (solo build): 2 admisibles de 3; 3 de 3 estrategias tienen al menos una pareja
admisible
  caen por pearson_monthly: 1
  caen por pearson_daily: 1
  caen por spearman_monthly: 1
  …
N efectivo: calma 1.9 / 1.8, estrés 1.5 / 4.0 (autovalores / media de ρ)
```

Las prohibiciones de una empresa (`--firm`)

Una estrategia que rompe una regla de un plan (cerrar el viernes, no operar en noticias…) **se quita del pool de ese plan; no se modifica** (regla tuya, encargo 33 §6.2). Quién rompe qué lo escribe el encargo 33 en `AlgoData/portfolio/prohibitions/<empresa>.csv`; el formato está en el `README.md` de esa carpeta. Hoy no hay ninguna prohibición real escrita: la carpeta solo tiene el formato. Salida real del 2026-10-05 en la raíz de prueba, con un archivo **inventado** para probar el flag:

```
Universo del pool 'Test_nikkei_ref' (65ed4e303bb7…)
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9,
'rolling_whole': 0.9, 'rolling_recent': 0.9}
Prohibiciones de hantec: 2 de 10 miembros fuera del pool antes de buscar
  prohibida 189a7efb30eb…: weekend (todos los planes) — PRUEBA de la fase 2: prohibición
inventada para probar --firm (no es una regla real)
  prohibida 4806ad307e7c…: news (hantec:express) — PRUEBA de la fase 2: prohibición inventada
para probar --firm (no es una regla real)
Miembros: 8 conservados, 0 excluidos de 8
Calendario: build 2011-09-19…2019-12-31, oos1 2020-01-01…2023-12-31, oos2 2024-01-01…2026-09-30
Relojes sin confirmar: ['hantec']
```

Un pool filtrado es otro pool: tiene otro hash y su propia carpeta de universo.

Qué NO te dice

- **Que dos estrategias independientes pasen**: con tus umbrales, solo el 75 % de las parejas de estrategias **independientes** pasan todos los filtros sobre 10 años de build (las ventanas móviles y el co-loss rechazan por azar). Para K estrategias independientes, que pasen todas sus parejas es 0,75 elevado al número de parejas: 4 → 18 %, 6 → 1,5 %. Una cartera pequeña puede ser ruido de los filtros, no redundancia.
- **El swap de las noches**: SQX lo cobra al cerrar; el flotante de las noches no lo lleva.
- **El N efectivo por «media de ρ»** puede pasar de K cuando las correlaciones son negativas (4,0 con 3 estrategias arriba): léelo junto al de autovalores.
- Nada de oos1 ni oos2: las parejas se miden solo en build.

Si algo falla

- `excluida · motivo: …`: esa estrategia no cuadra con SQX; mírala antes de usarla.
- Un pool que no existe: `FileNotFoundError` con la ruta del CSV.
- Falta el M1 de un feed: `FileNotFoundError` en `AlgoData/bars/<feed>/M1.parquet`.

79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan

Qué pregunta responde

Para un plan de fondeo concreto (p. ej. Hantec Enhanced de 10k): de todas las carteras que el cribado de parejas permite, ¿cuál aprueba el challenge con más probabilidad en **6 meses** si la hubieras comprado cualquier día del build, y a qué riesgo por operación? Es la etapa A del camino de fondeo (hito F2 de `portfolio/PLAN.md`): elige candidatas mirando solo build. El juicio fuera de muestra (oos1 + oos2) y el Monte Carlo vienen después (F3, F4).

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool cuyas estrategias llevan ya el **stop del paso 24** injertado. **No** te dice si la cartera es buena fuera de muestra, ni si el plan te sale rentable: eso es F4 y la economía del banco (F5).

Antes de empezar

- Un pool declarado (capítulo 77) cuyos miembros sean las versiones **con stop** del paso 24 (SL = X·ATR(20), X = p90), archivadas con su propio retest. Una estrategia sin ese stop queda **fuera** y se dice.
- **Un ajuste no tiene valor por defecto y el comando se para si no lo das**: `funded.open_risk_cap`, el tope de riesgo abierto (abajo). La regla es tuya (2026-10-05); la fracción todavía no la has fijado.
- Nada de SQX. Usa todos los núcleos de la máquina y hasta 80 GB de RAM: **no lo lances mientras el custodio corre un trabajo largo** (los dos no caben juntos).

Cómo se ejecuta

```
python3 -m portfolio.common.construct.funded --pool mi_pool --plan hantec:enhanced:10000:USD \
  --firm hantec --set funded.open_risk_cap=0.5
```

flag	obligatorio	qué hace
`--pool`	sí	el pool declarado
`--plan`	sí	el plan, con la clave del catálogo (`python3 -m portfolio.funded.catalog.show hantec`)
`--set funded.open_risk_cap=…`	sí	la fracción del tope de riesgo abierto, abajo
`--firm`	no	la empresa del plan, cuyas prohibiciones se aplican (`AlgoData/portfolio/prohibitions/<empresa>.csv`): las estrategias que esa empresa prohíbe salen del pool **antes** de buscar y se nombran una a una; cuentan las prohibiciones de toda la empresa y las de ese plan (capítulo 78). Sin el flag no se aplica ninguna
`--set`	no	cambia un ajuste de `config.yaml`, p. ej. `--set funded.horizon_days=63`

Qué produce

`AlgoData/portfolio/runs/<fecha>_<pool>_<plan>/`: `search.parquet` (cada cartera evaluada, con sus miembros, su P(aprobar), su riesgo y de dónde salió) y `manifest.json`. Además, **filas en el ledger**: una en el estudio `PORTFOLIO_<pool>_construct` y una en el estudio de cada miembro, con todas las carteras probadas — cada una cuenta como un intento para el Sharpe deflactado.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Riesgo fijo**: cada operación arriesga r % del saldo **inicial** con su stop X·ATR(20). SQX simuló con 1.000 $ sobre 4·ATR(20); el motor lo convierte con una constante por estrategia.
- **Rejilla**: r = 0,1 %, 0,2 % … 1,0 %, igual para todas las estrategias de la cartera. Un nivel en el que alguna operación quedaría por debajo de 0,01 lotes no es factible.
- **P(aprobar)**: de todos los días de build en los que podrías haber comprado (con 6 meses por delante), qué parte aprueba todas las fases en ≤ 126 días de trading. Las reglas se miran sobre la equity **flotante** del día del servidor de la empresa, con el peor momento del día desde M1.
- **Tope de riesgo abierto** (regla tuya, 2026-10-05): número de estrategias × riesgo por operación ≤ `funded.open_risk_cap` × la pérdida diaria máxima del plan. Con 0,5 y un plan del 5 % diario, una cartera de 4 no puede arriesgar más de 0,6 % por operación (4 × 0,6 = 2,4 ≤ 2,5). Lo que pasa del tope no es factible y la salida dice cuántos pares (cartera, riesgo) ha prohibido. No hay tope por símbolo. **Dos lecturas que ha tomado el código y tienes que confirmar**: «pérdida diaria máxima» es el porcentaje del catálogo sobre el saldo **inicial**, sea cual sea la base diaria del plan; y en un plan de varias fases manda la fase con la pérdida diaria más estricta.
- **Mismo riesgo en todas las fases** aquí; el riesgo por fase se barre en la etapa B.

La salida lista las 10 mejores: `K` (cuántas estrategias), `P(pass)`, `r` y quiénes son. Una regla sin confirmar del catálogo o el reloj de Hantec salen nombrados.

Un ejemplo completo

Salida real del 2026-10-05, la primera sobre un pool con el stop del paso 24: `Test_nikkei_ref`, diez estrategias del Nikkei casi iguales, en la **raíz de prueba**, con los umbrales de parejas relajados a 0,90 y el tope pasado a mano. **No es un resultado**: sirve para ver que el comando funciona.

```
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
Pool 'Test_nikkei_ref' plan hantec:express:10000:USD: 10 miembros en el universo
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9,
'rolling_whole': 0.9, 'rolling_recent': 0.9} — no es un resultado
Sin --firm: no se ha aplicado ninguna prohibición de empresa
Búsqueda: 4500 combinaciones evaluadas (58 distintas) en 15 generaciones
Tope de riesgo abierto: miembros × riesgo ≤ 0.5 × pérdida diaria máxima 5.0% = 2.50% del
balance inicial; prohíbe 14034 de 45000 pares (cartera, riesgo) evaluados
  K=3 P(pass)=0.338 r=0.600% [189a7efb, 4806ad30, e1cbf1bc]
  K=3 P(pass)=0.338 r=0.600% [189a7efb, 4806ad30, 7f37f07c]
  K=3 P(pass)=0.334 r=0.600% [189a7efb, 95656395, c9b701e7]
  K=3 P(pass)=0.333 r=0.600% [189a7efb, 4806ad30, 95656395]
  K=3 P(pass)=0.333 r=0.600% [189a7efb, 4806ad30, c9b701e7]
  K=4 P(pass)=0.328 r=0.600% [189a7efb, 4806ad30, c29a3efb, c9b701e7]
  K=2 P(pass)=0.325 r=0.800% [4806ad30, c9b701e7]
  K=2 P(pass)=0.322 r=0.800% [4806ad30, e7550f9c]
  K=2 P(pass)=0.320 r=0.900% [189a7efb, 4806ad30]
  K=3 P(pass)=0.319 r=0.800% [4806ad30, c29a3efb, c9b701e7]
Tiempo: 13.5 s (GA propio ~13.3 ms/generación, evaluación 1.8 s) · RAM pico: 1099 MB
Reglas sin confirmar: ['daily_loss_basis', 'max_loss_trailing_order']
Carpeta: <raíz de prueba>/portfolio/runs/2026-10-
05T1000_Test_nikkei_ref_hantec_express_10000_USD
```

Cómo leerlo: de 4.500 evaluaciones del genético solo 58 carteras son distintas (el pool admite 103): con un pool tan pequeño repite. La mejor aprueba el 33,8 % de las veces con 3 estrategias al 0,6 %. El tope cambia la respuesta según la fracción — medido el mismo día sobre el mismo pool:

`funded.open_risk_cap`	pares prohibidos	la mejor
1,0	0 de 45.000	K=3 al 0,6 %, P 0,338
0,5	14.034 de 45.000	la misma: el tope no la toca
0,25	26.533 de 48.000	cambia: K=2 al 0,6 %, P 0,181

En `ftmo:2step:10000:USD` con 0,5 la mejor es K=3 al 0,8 %, P 0,166 (3,6 s). Con `--firm hantec` y el archivo de prohibiciones inventado para la prueba, salen dos estrategias antes de buscar y la mejor pasa a ser otra (K=2, P 0,300).

El stop del paso 24 se lee de verdad (`tests/test_portfolio_stopread.py`): con el stop de cada estrategia multiplicado por 2 y las mismas operaciones, la misma cartera pasa de P 0,339 a 0,102 al 1 % de riesgo.

Rendimiento: 13,5 s y 1,1 GB con el universo en caché; medido antes en datos sintéticos (50 estrategias, 10 años), ~1.500 carteras por segundo en 48 núcleos.

Qué NO te dice

- **Nada fuera de muestra**: elige en build; P(aprobar) en build está inflada por la propia búsqueda.

- **No hay Monte Carlo** todavía: es el camino real de la historia desde cada día de inicio.
- **El redondeo de lotes** no se simula aquí (solo se comprueba que ninguno quede en cero).
- **Una cartera repetida por el genético cuenta dos veces** en el ledger: el número de intentos sale por exceso, nunca por defecto.

Si algo falla

- `funded.open_risk_cap no está fijado`: pásalo con `--set funded.open_risk_cap=<fracción>`.
- `--firm X no es la empresa del plan`: el flag tiene que ser la empresa del plan que pides.
- `sin stop del paso 24`: ese miembro es la versión original; archiva la versión con stop.
- `NotImplementedError` con `min_days`: el catálogo no sabe los días mínimos de ese plan; el fundingWatcher o tú tenéis que confirmarlo.

87. Cartera · la etapa B y el veredicto de fondeo — ¿aguanta la cartera elegida un Monte Carlo, y fuera de muestra?

Qué pregunta responde

La búsqueda de fondeo (capítulo 79) te deja miles de carteras ordenadas por su probabilidad de aprobar un plan mirando la historia real de build. Aquí se cogen **las mejores, distintas entre sí**, y se les hace la prueba dura: tres Monte Carlo distintos, con el riesgo **por fase**. Sale una ganadora por plan. Después, el veredicto, en dos pruebas que **no se mezclan** (decisión del Council del 2026-10-04, Q11 d): esa ganadora, leída en **oos1**, ¿queda por encima de lo que saca una búsqueda igual hecha sobre carteras al azar del mismo pool? Y solo si pasa, **oos2, una sola vez**: ¿sigue siendo positivo su Sharpe después de descontar todo lo que se ha probado?

Son los hitos F3 (etapa B) y F4 (veredicto) de `portfolio/PLAN.md` §14.

Cuándo lo usas, y cuándo no

Después de una búsqueda de fondeo (capítulo 79) que haya dejado su carpeta en `AlgoData/portfolio/runs/`. **No** lo uses para decidir cuánto dinero meter ni cuántas cuentas comprar: eso es la simulación del banco (F5). **No** emite veredicto si el universo no tiene días en oos1. Y cuidado: si pasa oos1 **abre oos2, que se lee una vez por pool y nunca más** — por esta vía o por la de la cuenta real (capítulo 101), la que llegue antes.

Antes de empezar

- La carpeta de una búsqueda de fondeo ya hecha, con su `search.parquet` y su `manifest.json`.
- Las estrategias del pool, archivadas con su stop del paso 24, y las barras M1 de sus mercados.
- Nada de SQX. Usa todos los núcleos y hasta 80 GB: **no lo lances con el custodio trabajando**.
- **Dos ajustes no tienen valor por defecto y el comando se para si no los das** (son decisiones tuyas, todavía abiertas):
- `funded.open_risk_cap`: el tope de riesgo abierto (capítulo 79). Aquí se aplica **por fase**: cada fase con su riesgo y su pérdida diaria.
- `stageb.t1_gap`: cómo se separan las operaciones en el Monte Carlo T1. Solo hay una forma construida, `close_to_open_blocks` (ver abajo); se te pide que la aceptes a propósito.
- «No dos finalistas comparten más de la mitad de sus miembros» es la mitad de la cartera **más grande** (`stageb.shared_of: larger`, decisión tuya del 2026-10-05).
- La empresa, sus prohibiciones y la marca AJUSTADO se heredan de la búsqueda de la etapa A: aquí no hay `--firm`.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.funded_b \
  --run ~/Desktop/AlgoData/portfolio/runs/2026-10-02T1405_mi_pool_hantec_enhanced_10000_USD \
  --set funded.open_risk_cap=0.5 --set stageb.t1_gap=close_to_open_blocks
```

flag	obligatorio	qué hace
`--run`	sí	la carpeta de la búsqueda de fondeo (etapa A) sobre la que se trabaja
`--set funded.open_risk_cap=…`	sí	la misma fracción que en la etapa A
`--set stageb.t1_gap=close_to_open_blocks`	sí	acepta el modelo de huecos de T1
`--set stageb.shortlist_size=50`	no	cuántas finalistas (50)
`--set stageb.n_paths=10000`	no	caminos por Monte Carlo y longitud de bloque (10.000)
`--set verdict.null_draws=100`	no	cuántas búsquedas al azar forman el nulo (100)
`--set verdict.null_candidates=0`	no	carteras al azar por búsqueda nula; 0 = tantas como evaluó la etapa A

Si la búsqueda de la etapa A se lanzó con algún `--set` (por ejemplo otro horizonte), pásale aquí el mismo. No toca SQX.

Cuánto tarda (medido el 2026-10-02 en 8 hilos y con la máquina ocupada): una finalista de 10 estrategias, a tamaño completo (10.000 caminos × 11 juegos de caminos × 100 riesgos), 17 segundos; 50 finalistas, unos 14 minutos. **El veredicto repite la etapa B una vez por cada búsqueda nula**: medido el 2026-10-05 sobre 10 estrategias, unos 22 s por búsqueda nula con la máquina libre y 35-39 s con ella ocupada (20 búsquedas: 12-13 minutos; las 100, entre 40 minutos y una hora). En un pool grande son horas. Es el veredicto, no la etapa B, lo que pide la máquina entera.

Qué produce

En la misma carpeta de la búsqueda — salvo que esa run ya tuviera una etapa B: entonces la nueva se escribe en una run aparte (copia de la etapa A, con `stage_a_run` en su manifiesto) y la anterior no se toca; el comando dice cuál es:

archivo	qué es
`shortlist.parquet`	una fila por finalista: sus miembros, su nota de la etapa A, su P(aprobar) de la etapa B, el riesgo por fase con que se queda, qué Monte Carlo decidió y con qué bloque, y la cifra de cada uno de los tres
`stageb.parquet`	**todas** las cifras: por finalista, Monte Carlo, longitud de bloque y riesgo — P(aprobar), P de cada fase, días hasta aprobar, qué regla rompe
`verdict.json`	la ganadora, la prueba de estabilidad, el veredicto (o por qué no se emite), cada búsqueda nula con sus cifras, la semilla y el coste
`verdict_rows.parquet`	las cifras de la ganadora en oos1 (solo si hay veredicto)
`handoff/`	la cartera para quien venga después (solo si hay veredicto): `portfolio.json`, `daily.parquet`, `trades.parquet`

Además, **filas en el ledger**: la etapa B una búsqueda por plan (finalistas × riesgos, todas cuentan como intentos); el veredicto, una por cada segmento que abre (oos1 siempre; oos2 solo si oos1 pasa). Cada fila va al estudio del pool y al de cada estrategia. La fila de oos2 guarda la lectura entera: es lo que impide leerlo dos veces.

Lanzarlo otra vez sobre la misma run no sobrescribe nada (desde el 2026-10-05): si la carpeta ya tiene `shortlist.parquet`, `stageb.parquet`, `verdict.json` o `verdict_rows.parquet`, el comando copia la etapa A a una run nueva y escribe allí; lo dice en su primera línea («Esta run ya tenía etapa B: la nueva queda en una run aparte, …»). La etapa B anterior se queda como estaba.

Cómo se lee el resultado

Las finalistas. De todas las carteras distintas de la etapa A se recorren de mejor a peor y se queda una si no comparte más de la mitad de sus miembros con ninguna de las ya elegidas.

Los tres Monte Carlo — cada uno fabrica 10.000 «semestres» posibles (126 días de servidor) y pasa las reglas del plan por cada uno:

nombre	qué sortea	qué conserva	qué pierde
D	rachas de días seguidos de la cartera entera	que las estrategias pierdan a la vez; el peor momento exacto de cada día (de M1)	una operación abierta al final de una racha se liquida ahí a su valor
T1	las operaciones de cada estrategia, con reposición, una tras otra con un hueco sorteado de su propia historia; cada estrategia por su cuenta	cada operación entera, con su flotante	que pierdan a la vez: lee menos caída de la real
T2	ventanas de calendario; todas las operaciones de todas las estrategias que se abrieron en la ventana, juntas	cada operación entera **y** qué se abrió a la vez	variedad: hay menos ventanas que operaciones

La longitud de las rachas no se elige, se mide: la que calcula Politis-White sobre el P&L diario de la propia cartera, y además 5, 10, 20 y 40 días. **Manda la más pesimista**, y entre los tres Monte Carlo **manda también el más pesimista** (tu regla del 2026-09-30). Todas las cifras se guardan.

El riesgo por fase. Cada fase del challenge prueba los 10 niveles de riesgo (0,1 % … 1,0 % por operación): un plan de dos fases son 100 combinaciones. La cartera se queda con la combinación cuya cifra pesimista es más alta.

Las cifras de cada fila:

- `P(aprobar)`: de los 10.000 caminos, cuántos aprueban todas las fases dentro del horizonte.
- `p_stage1`, `p_stage2`…: cuántos pasan la fase 1, cuántos la 2.
- `t_median`, `t_p90`: días de servidor desde la compra hasta aprobar, **solo entre los caminos que aprueban**. Los que llegan al final del horizonte sin aprobar ni romper van aparte, en `p_open`.
- `fail_daily_loss`, `fail_max_loss`, `fail_time_limit`: qué parte de los caminos rompe cada regla.

La estabilidad. La cifra que decide se recalcula 8 veces con sorteos nuevos. Si entre la mayor y la menor hay más de 2 puntos, lo dice: sube `stageb.n_paths` antes de decidir nada.

El veredicto. Se repite la misma búsqueda — misma nota de la etapa A, mismas finalistas diversas, misma etapa B — pero sobre **carteras admisibles al azar** del mismo pool, 100 veces. Cada búsqueda al azar da su propia ganadora. Luego la tuya y las 100 se leen **solo en oos1** con dos medidas, y se dice **en qué percentil queda la tuya**:

- *P(aprobar) en el camino real, desde cada día de inicio*: comprando el plan cada día de oos1, qué parte de las veces aprueba.
- *P(aprobar) del Monte Carlo más pesimista*: lo mismo que la etapa B, con los datos de oos1. **Esta es la que decide** (`verdict.pass_measure: mc_p`, decisión tuya del 2026-10-05); la salida la marca con «← la que decide».

Falta el listón. Qué percentil hay que superar sigue sin fijar (`verdict.pass_percentile`). Hasta que lo fijes, la última línea dice «SIN CRITERIO (falta verdict.pass_percentile)», te da las cifras y **oos2 no se abre**.

oos2, una vez. Con listón y oos1 pasado, se lee el Sharpe diario de la cartera en oos2 y se le resta **lo que el azar da como mejor de N intentos** — N es todo lo probado en el proyecto, σ la dispersión de los Sharpes de build de las carteras de este pool. Si queda por encima de cero, PASA. Se apunta en el registro pase o no, y el segmento queda gastado para el pool: otra cartera del mismo pool ya no puede leerlo.

Y «¿se sostuvo la diversificación?»: las correlaciones de las parejas de la cartera, medidas otra vez en los tramos que el veredicto abrió, contra los mismos umbrales. Solo se informa.

Un ejemplo completo

Sobre estrategias de verdad (2026-10-05): la búsqueda del capítulo 79 sobre `Test_nikkei_ref`, en la raíz de prueba, con umbrales relajados, un oos2 de mentira desde 2022 y solo 20 búsquedas nulas — **no es un resultado**. 12 minutos, 2,0 GB:

```
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
Etapa B · pool 'Test_nikkei_ref' plan hantec:express:10000:USD: 11 finalistas diversas de 58 carteras distintas (4500 evaluadas en la etapa A) × 10 riesgos
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9, 'pseudo_oos2_from': '2022-01-01'} — no es un resultado
  K=2 A=0.299 B=0.299 r=1.0% decide T2@2 · D 0.312 T1 0.377 T2 0.299 [45bc9b7b, 4806ad30]
  K=4 A=0.328 B=0.298 r=0.6% decide T2@2 · D 0.309 T1 0.363 T2 0.298 [189a7efb, 4806ad30, c29a3efb, c9b701e7]
  K=2 A=0.325 B=0.297 r=0.9% decide T2@5 · D 0.300 T1 0.346 T2 0.297 [4806ad30, c9b701e7]
  K=2 A=0.302 B=0.296 r=1.0% decide T2@3 · D 0.300 T1 0.382 T2 0.296 [4806ad30, 95656395]
  K=2 A=0.322 B=0.293 r=1.0% decide T2@2 · D 0.305 T1 0.386 T2 0.293 [4806ad30, e7550f9c]
  K=3 A=0.334 B=0.291 r=0.8% decide D@40 · D 0.291 T1 0.448 T2 0.293 [189a7efb, 95656395, c9b701e7]
  K=3 A=0.338 B=0.281 r=0.7% decide T2@5 · D 0.297 T1 0.346 T2 0.281 [189a7efb, 4806ad30, e1cbf1bc]
  K=3 A=0.292 B=0.278 r=0.7% decide T2@1 · D 0.288 T1 0.326 T2 0.278 [189a7efb, 7f37f07c, e7550f9c]
  K=4 A=0.249 B=0.274 r=0.6% decide T2@40 · D 0.281 T1 0.318 T2 0.274 [189a7efb, 5b9b06ce, c29a3efb, e1cbf1bc]
  K=4 A=0.280 B=0.274 r=0.5% decide T2@20 · D 0.286 T1 0.278 T2 0.274 [189a7efb, 4806ad30, 5b9b06ce, 7f37f07c]
Ganadora: P(aprobar) 0.296 en caminos nuevos (la búsqueda la eligió con 0.299, el mejor de muchos sorteos) con el Monte Carlo más pesimista (T2, bloque 2); tarda 66 días de mediana (p90 112) entre las que aprueban; 19.8% siguen abiertas al acabar el horizonte; rompe pérdida diaria 3.7%, pérdida máxima 46.6%, plazo 0.0%
Estabilidad: 8 repeticiones, dispersión 0.010 — los caminos sorteados bastan
Reglas sin confirmar: ['daily_loss_basis', 'max_loss_trailing_order']
Reloj del servidor de hantec sin confirmar: el día se corta donde dice config.yaml (equity.firm_clocks)
La cartera de 2 estrategias (de 10 admisibles; 4500 combinaciones probadas en la etapa A, 11 finalistas × 10 riesgos en la etapa B), a riesgo 1.0% por fase, leída en oos1 contra 20 búsquedas al azar de 4500 combinaciones cada una:
  oos1 · P(aprobar) en el camino real, desde cada día de inicio: 0.226 — percentil 15% del nulo seleccionado (mediana nula 0.257, p95 0.257)
  oos1 · P(aprobar) del Monte Carlo más pesimista: 0.163 — percentil 35% del nulo seleccionado (mediana nula 0.195, p95 0.202) ← la que decide
  0 de las 20 búsquedas al azar eligieron esta misma cartera
  oos2: no se lee — oos1 no se ha pasado: oos2 no se abre
  diversificación en oos1: 0 de 1 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: SIN CRITERIO (falta verdict.pass_percentile) — no se aprueba ni se descarta nada.
```

Cómo leerlo: los tres Monte Carlo quedan cerca de la nota A. **T1 sale casi siempre el más alto**: sortea cada estrategia por separado, así que las pérdidas que en la historia coincidían dejan de coincidir; con estrategias casi iguales eso es casi todo el riesgo. Por eso decide el más pesimista. En oos1 la ganadora aprueba el 16,3 % de las veces según el Monte Carlo, por debajo de la mediana de las búsquedas al azar (19,5 %): percentil 35 %. Como no hay listón, la decisión es «SIN CRITERIO» y oos2 sigue cerrado.

Desde el 2026-10-04 la línea «Ganadora» da **dos** cifras: «P(aprobar) X en caminos nuevos (la búsqueda la eligió con Y, el mejor de muchos sorteos)». X es la que vale: se recalcula sobre caminos que la búsqueda no vio. En `verdict.json`, X es `build_p` e Y es `selector_p`.

Con un listón de prueba (`--set verdict.pass_percentile=0.0`, que deja pasar a cualquiera: solo sirve para ver funcionar la segunda prueba) el final de la salida es:

```
La cartera de 2 estrategias (de 10 admisibles; 4500 combinaciones probadas en la etapa A, 11 finalistas × 10 riesgos en la etapa B), a riesgo 1.0% por fase, leída en oos1 contra 20 búsquedas al azar de 4500 combinaciones cada una:
  oos1 · P(aprobar) en el camino real, desde cada día de inicio: 0.275 — percentil 100% del nulo seleccionado (mediana nula 0.257, p95 0.257)
  oos1 · P(aprobar) del Monte Carlo más pesimista: 0.202 — percentil 85% del nulo seleccionado (mediana nula 0.197, p95 0.203) ← la que decide
  0 de las 20 búsquedas al azar eligieron esta misma cartera
  oos2: Sharpe diario 0.0015 menos el máximo que da el azar 0.0273 (N = 383,600 pruebas de todo el proyecto, σ = 0.0058) = -0.0258 en 503 días; DSR 0.28 → NO PASA. Leído una vez: el segmento queda gastado para este pool.
  diversificación en oos1+oos2: 0 de 1 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: NO PASA (oos2) — no se compra el plan con esta cartera (mc_p en el percentil 85% en oos1, listón 0%).
```

El Sharpe diario en oos2 (0,0015) no llega a lo que el azar da como mejor de 383.600 intentos (0,0273): NO PASA, y el oos2 de ese pool de prueba queda gastado.

Y al lanzarlo otra vez, la etapa B eligió otra ganadora (las finalistas están a milésimas unas de otras y los sorteos son nuevos), y el oos2 ya no se abre para ella:

```
La cartera de 4 estrategias (de 10 admisibles; 4500 combinaciones probadas en la etapa A, 11 finalistas × 10 riesgos en la etapa B), a riesgo 0.6% por fase, leída en oos1 contra 3 búsquedas al azar de 4500 combinaciones cada una:
  oos1 · P(aprobar) en el camino real, desde cada día de inicio: 0.226 — percentil 0% del nulo seleccionado (mediana nula 0.257, p95 0.257)
  oos1 · P(aprobar) del Monte Carlo más pesimista: 0.078 — percentil 0% del nulo seleccionado (mediana nula 0.197, p95 0.203) ← la que decide
  0 de las 3 búsquedas al azar eligieron esta misma cartera
  oos2: no se lee — el oos2 de este pool ya lo leyó otra cartera el 2026-10-05: es una sola lectura, de una sola cartera, y está gastada
  diversificación en oos1: 0 de 6 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: SIN VEREDICTO: oos1 pasado, oos2 sin leer (el oos2 de este pool ya lo leyó otra cartera el 2026-10-05: es una sola lectura, de una sola cartera, y está gastada) — no se aprueba ni se descarta nada (mc_p en el percentil 0% en oos1, listón 0%).
```

Si falta un ajuste obligatorio, se para sin leer nada:

```
funded.open_risk_cap no está fijado: qué fracción de la pérdida diaria máxima del plan puede estar en riesgo a la vez (miembros × riesgo por operación). No lleva valor por defecto: --set funded.open_risk_cap=<fracción>
```

Qué NO te dice

- **El percentil no dice si las estrategias tienen ventaja.** El nulo sale del mismo pool, así que la ventaja del pool está también en el nulo. Dice si **la búsqueda eligió mejor que el azar entre carteras admisibles**. En un pool pequeño, una búsqueda al azar con muchas candidatas encuentra la misma cartera y el percentil sale ~50 % aunque la cartera sea buena (medido: 100 % de las búsquedas nulas eligieron la misma pareja, percentil 0,50).
- **La P(aprobar) de la etapa B sigue siendo de build**: está inflada por haber elegido. La que vale es la de oos1.
- **La ganadora cambia de un lanzamiento a otro** cuando las finalistas están muy juntas (visto en el pool de prueba: tres lanzamientos, tres ganadoras). Después de leer oos2, relanzar puede elegir otra cartera, y esa ya no tiene oos2.
- **Un pool que solo ha pasado por la búsqueda de fondeo no puede leer oos2**: el descuento del Sharpe usa los Sharpes de build que escribe la búsqueda por rangos (capítulo 96).
- **T1 no conserva la hora**: cada operación cae donde la deje el hueco sorteado. Sirve para la suerte del orden, no para lo que pasa entre estrategias.
- **Una operación a caballo del primer o el último día del tramo** no entra en T1 ni T2 (sí en D).
- **El redondeo de lotes no se simula**; solo se descarta el riesgo en que algún lote quedaría en cero.
- **Nada de dinero**: ni precio del plan, ni recompras, ni cobros. Eso es el banco (F5).

Si algo falla

- `funded.open_risk_cap no está fijado` / `stageb.t1_gap no está fijado`: pásalos con `--set`.

- Termina con «Decisión: NINGUNA todavía — sin oos1 en el universo»: las estrategias del pool no tienen días en oos1. La etapa B sí se ha escrito; el veredicto espera a esos datos.
- «SIN CRITERIO (falta verdict.pass_percentile)»: no es un fallo; falta tu listón.
- «el oos2 de este pool ya lo leyó otra cartera»: está gastado, no hay segunda lectura.
- `T1: los miembros […] tienen menos de 2 operaciones`: una estrategia casi no opera en el tramo y no hay huecos que sortear; no puede entrar en una cartera de fondeo.
- Una estrategia que estaba en el pool desaparece («licencia MAE/MFE»): está archivada con el reloj del feed antiguo y las barras de hoy están en otro (cambio del 2026-10-01). Hay que volver a cosecharla sobre el feed nuevo.

88. La caja del fondeo — lo que sale de tu banco contra lo que vuelve, plan por plan

Qué pregunta responde

Si compro este challenge con esta cartera y a este riesgo, **¿entra más dinero en mi cuenta del banco por cobros del que sale por compras?** No mira la rentabilidad de la cuenta fondeada: mira la caja real. Cuenta cada compra (también las recompras tras cada suspenso), cada cobro con su reparto, y el reembolso de la cuota donde la empresa lo da:

```
VE_banco = lo que esperas cobrar − lo que esperas pagar
```

Para cada plan de tu universo (cuentas de 10k como mucho, en USD, de las empresas activas) te da una fila con el riesgo por fase que más rinde, la probabilidad de aprobar cada fase y de llegar al primer cobro, cuántas compras hacen falta, cuánto dura y cuánto paga una fondeada, el VE por compra, por dólar y por mes, **cuánto dinero tienes que adelantar antes del primer cobro**, cuánto edge puedes perder antes de que la caja pierda, y cuánto de todo eso lo ganaría un mono.

Cómo lo simula

Coge un camino largo de la cartera (años de días del servidor de la empresa, con la equity flotante de cada día) y **compra las cuentas una detrás de otra sobre ese mismo camino**: compra el día 0 → fases del challenge → fondeada → cobro cada N días → una regla la rompe → al día siguiente recompra… hasta que se acaba el camino o la caja. Así dos cuentas seguidas viven el mismo régimen de mercado y suspenden juntas, como pasa de verdad. Eso se repite sobre miles de caminos.

- **El presupuesto es un tope duro**: tu cuenta del banco tiene 1.000 €; cada cobro vuelve a ella; una compra sólo se hace si queda dinero para pagarla.
- **El recorte** `h` quita esa fracción del edge de la cartera (h = 50 % → la cartera rinde en vivo la mitad que en el OOS). La tabla enseña la curva entera — 0, 25, 50, 75 y 100 % — y se ordena con el 50 % (lo fijaste el 2026-10-05).
- **La recompra se compara** (también del 2026-10-05): cada plan sale dos veces, con `same_plan` (tras un suspenso recompras el mismo plan mientras quede caja) y con `none` (una sola cuenta, y lo peor que puede pasar es perder la cuota). Como la política es una cosa más que se busca, el número de pruebas que se apunta en el ledger es el doble.
- **El mono** es la misma cartera sin edge: mismos días, misma volatilidad, deriva cero. Lo que gana el mono es lo que vale el plan como lotería; lo que aportan tus EAs es la **ventaja** sobre él.
- **Los add-ons** se evalúan como ΔVE: el VE con el add-on menos el VE sin él. El precio del add-on ya está descontado (se paga en cada compra).

Cuándo lo usas, y cuándo no

Cuando el motor de carteras ya ha elegido y juzgado una cartera fondeada y quieres saber qué plan comprarle. **No** elige la cartera (eso es del motor, capítulo 79). Este comando hace la tabla; la frase «compra esto» o «ninguno» la da el capítulo 102 leyendo esta tabla.

No sirve con caminos que no vengan del pool validado de esa empresa, ni para comparar empresas cuyas reglas siguen sin confirmar sin leer antes la lista «Sin confirmar» que imprime al final.

Antes de empezar

No toca SQX ni MetaTrader. Necesita:

1. **Un conjunto de caminos** (`.npz`) de la cartera que el veredicto fondeado eligió (capítulo 87). Lo fabrica el primer comando de «Cómo se ejecuta». Lleva la tabla de días del servidor de cada camino (`closed`, `float_end`, `low`, `high`, `opened`), `unit_risk` (cuánto se arriesga por operación en esas cifras), `days_per_year`, y de dónde sale (corrida, cartera, veredicto, edge).
2. **Tus decisiones**, que el comando no supone nunca. Sin ellas se niega y te dice cuál falta:

clave	qué es
`owner.eurusd`	el cambio EUR/USD del día (dólares por euro). No se descarga: lo das tú y queda escrito con el resultado
`owner.payout_cushion`	fracción del tamaño que dejas en la cuenta en cada cobro; 0 = retirar todo
`owner.tax_rate`	fracción de cada cobro que se va en impuestos
`owner.payout_fee`	fracción de cada cobro que se va en comisiones de pago y divisa
`owner.p_payout_denied`	probabilidad de que la empresa deniegue un cobro (la cuenta se acaba ahí)
`owner.p_firm_closes_year`	probabilidad de que la empresa cierre en un año
`gaps.hantec_endurance_payout_days`	cada cuántos días cobra una Endurance fondeada. El catálogo no lo trae
`gaps.hantec_express_floor_after_payout`	qué pasa con la pérdida máxima de la Express fondeada tras un cobro: `stays` (el retiro no la mueve: queda donde la dejó el máximo alcanzado) o `initial` (desde el primer retiro queda fija en el balance inicial)
`gaps.ftmo_1step_floor_after_payout`	lo mismo para la FTMO 1-Step fondeada

Ya están puestos, y no hay que darlos: el presupuesto (`owner.budget_eur`, 1.000 €, 2026-09-29), la recompra (`owner.rebuy: [same_plan, none]`, las dos comparadas), los recortes (`owner.haircuts`: 0, 25, 50, 75 y 100 %) y el que decide (`owner.haircut_decides`: 50 %), los tres del 2026-10-05.

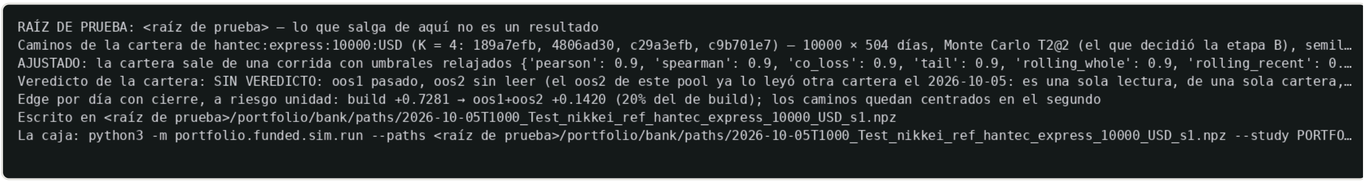
Cómo se ejecuta

Dos pasos. **Primero los caminos** de la cartera que eligió el veredicto:

```
python3 -m portfolio.funded.sim.frompool \
  --run ~/Desktop/AlgoData/portfolio/runs/<corrida fondeada> \
  --horizon 504 --n-paths 10000 --edge-from oos1,oos2 \
  --set <los mismos --set con los que se hizo la corrida>
```

flag	obligatorio	qué hace
`--run`	sí	la carpeta de la corrida fondeada; tiene que tener `verdict.json` (capítulo 87)
`--horizon`	sí	cuántos días de servidor dura cada camino: **cuánto tiempo se simula tu banco**. No tiene valor por defecto porque cambia el resultado (ver «Qué NO te dice»)
`--n-paths`	sí	cuántos caminos; 10.000 da un error típico de 1-4 $ en el VE
`--edge-from`	sí	de qué tramos fuera de muestra sale el **nivel** del edge: los caminos se sortean del build (que da la forma: días, volatilidad, rachas) y su media se mueve a la de estos tramos
`--seed`	no	raíz del sorteo; sin él se sortea una y se imprime
`--out`	no	dónde escribir el `.npz`

Imprime la cartera, el Monte Carlo usado (el mismo que decidió la etapa B), el veredicto que tiene esa cartera, cuánto edge queda fuera de muestra respecto al build, y **la línea del segundo comando**:



```
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
Caminos de la cartera de hantec:express:10000:USD (K = 4: 189a7efb, 4806ad30, c29a3efb, c9b701e7) — 10000 × 504 días, Monte Carlo T2@2 (el que decidió la etapa B), semil…
AJUSTADO: la cartera sale de una corrida con umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.…
Veredicto de la cartera: SIN VEREDICTO: oos1 pasado, oos2 sin leer (el oos2 de este pool ya lo leyó otra cartera el 2026-10-05: es una sola lectura, de una sola cartera,…
Edge por día con cierre, a riesgo unidad: build +0.7281 → oos1+oos2 +0.1420 (20% del de build); los caminos quedan centrados en el segundo
Escrito en <raíz de prueba>/portfolio/bank/paths/2026-10-05T1000_Test_nikkei_ref_hantec_express_10000_USD_s1.npz
La caja: python3 -m portfolio.funded.sim.run --paths <raíz de prueba>/portfolio/bank/paths/2026-10-05T1000_Test_nikkei_ref_hantec_express_10000_USD_s1.npz --study PORTFO…
```

Después, la caja:

```
python3 -m portfolio.funded.sim.run \
  --paths ~/Desktop/AlgoData/portfolio/bank/paths/<corrida>_s<semilla>.npz \
  --study PORTFOLIO_abc123def456_construct --symbol NIKKEI225 --segment oos2 \
  --set owner.eurusd=1.17 \
  --set owner.payout_cushion=0.02 --set owner.tax_rate=0.19 --set owner.payout_fee=0.02 \
  --set owner.p_payout_denied=0.02 --set owner.p_firm_closes_year=0.05 \
  --set gaps.hantec_endurance_payout_days=14 \
  --set gaps.hantec_express_floor_after_payout=initial --set
gaps.ftmo_1step_floor_after_payout=stays
```

(Las cifras de arriba son un ejemplo de forma, no una recomendación.)

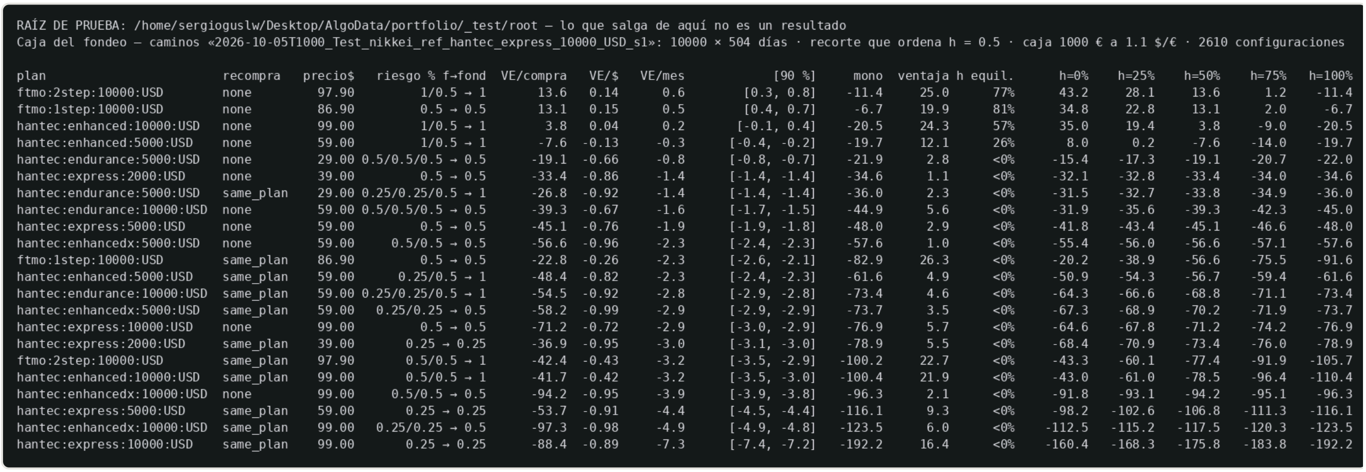
flag	obligatorio	qué hace
`--paths`	sí (o `--days`)	el conjunto de caminos del primer paso
`--days` + `--horizon --n-paths --mean-block --unit-risk --days-per-year`	alternativa	parte de una tabla de días y sortea los caminos en el momento, con el bootstrap por bloques de la etapa B
`--study --symbol --segment`	sí con caminos reales	el estudio del ledger donde se cuenta la búsqueda: cada plan × add-on × riesgo × política de recompra probado es una mirada (2.610 hoy)
`--monkey`	no	un conjunto de caminos del mono hecho aparte (entradas al azar de verdad, con costes); si no, el mono es la cartera sin deriva
`--seed`	no	raíz de los sorteos de cobro denegado y cierre; sin él se sortea una y se imprime
`--out`	no	carpeta de salida
`--set risk.challenge=[…]`, `--set risk.funded=[…]`	no	la rejilla de riesgo por operación (fracción del balance inicial) de las fases y de la fondeada
`--set addons.max_set=2`	no	evaluar los add-ons también por parejas

Los caminos tardan unos 12 segundos; la caja, minuto y medio con 10.000 caminos de 504 días (2.610 configuraciones, unas 33 milésimas cada una).

Qué produce

Los caminos, en `AlgoData/portfolio/bank/paths/`. La caja, en `AlgoData/portfolio/bank/<fecha>_<caminos>/`: `table.parquet` (la tabla, una fila por plan y política de recompra), `addons.parquet`, `search.parquet` (todas las configuraciones probadas con su VE), `result.json` (lo mismo en el formato que pinta la ventana) y `manifest.json` (los caminos, la semilla, **el cambio EUR/USD usado**, la configuración entera y la fila del ledger). No sobrescribe nada.

Cómo se lee el resultado



```
RAÍZ DE PRUEBA: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
Caja del fondeo — caminos «2026-10-05T1000_Test_nikkei_ref_hantec_express_10000_USD_s1»: 10000 × 504 días · recorte que ordena h = 0.5 · caja 1000 € a 1.1 $/€ · 2610 configuraciones

plan                        recompra   precio$   riesgo % f→fond  VE/compra   VE/$   VE/mes            [90 %]     mono  ventaja h equil.      h=0%    h=25%    h=50%    h=75%   h=100%
ftmo:2step:10000:USD        none         97.90         1/0.5 → 1       13.6   0.14      0.6        [0.3, 0.8]    -11.4     25.0      77%      43.2     28.1     13.6      1.2    -11.4
ftmo:1step:10000:USD        none         86.90         0.5 → 0.5       13.1   0.15      0.5        [0.4, 0.7]     -6.7     19.9      81%      34.8     22.8     13.1      2.0     -6.7
hantec:enhanced:10000:USD   none         99.00         1/0.5 → 1        3.8   0.04      0.2       [-0.1, 0.4]    -20.5     24.3      57%      35.0     19.4      3.8     -9.0    -20.5
hantec:enhanced:5000:USD    none         59.00         1/0.5 → 1       -7.6  -0.13     -0.3      [-0.4, -0.2]    -19.7     12.1      26%       8.0      0.2     -7.6    -14.0    -19.7
hantec:endurance:5000:USD   none         29.00 0.5/0.5/0.5 → 0.5      -19.1  -0.66     -0.8      [-0.8, -0.7]    -21.9      2.8      <0%     -15.4    -17.3    -19.1    -20.7    -22.0
hantec:express:2000:USD     none         39.00         0.5 → 0.5      -33.4  -0.86     -1.4      [-1.4, -1.4]    -34.6      1.1      <0%     -32.1    -32.8    -33.4    -34.0    -34.6
hantec:endurance:5000:USD   same_plan    29.00 0.25/0.25/0.5 → 1      -26.8  -0.92     -1.4      [-1.4, -1.4]    -36.0      2.3      <0%     -31.5    -32.7    -33.8    -34.9    -36.0
hantec:endurance:10000:USD  none         59.00 0.5/0.5/0.5 → 0.5      -39.3  -0.67     -1.6      [-1.7, -1.5]    -44.9      5.6      <0%     -31.9    -35.6    -39.3    -42.3    -45.0
hantec:express:5000:USD     none         59.00         0.5 → 0.5      -45.1  -0.76     -1.9      [-1.9, -1.8]    -48.0      2.9      <0%     -41.8    -43.4    -45.1    -46.6    -48.0
hantec:enhancedx:5000:USD   none         59.00     0.5/0.5 → 0.5      -56.6  -0.96     -2.3      [-2.4, -2.3]    -57.6      1.0      <0%     -55.4    -56.0    -56.6    -57.1    -57.6
ftmo:1step:10000:USD        same_plan    86.90         0.5 → 0.5      -22.8  -0.26     -2.3      [-2.6, -2.1]    -82.9     26.3      <0%     -20.2    -38.9    -56.6    -75.5    -91.6
hantec:enhanced:5000:USD    same_plan    59.00      0.25/0.5 → 1      -48.4  -0.82     -2.3      [-2.4, -2.3]    -61.6      4.9      <0%     -50.9    -54.3    -56.7    -59.4    -61.6
hantec:endurance:10000:USD  same_plan    59.00 0.25/0.25/0.5 → 1      -54.5  -0.92     -2.8      [-2.9, -2.8]    -73.4      4.6      <0%     -64.3    -66.6    -68.8    -71.1    -73.4
hantec:enhancedx:5000:USD   same_plan    59.00   0.25/0.25 → 0.5      -58.2  -0.99     -2.9      [-2.9, -2.9]    -73.7      3.5      <0%     -67.3    -68.9    -70.2    -71.9    -73.7
hantec:express:10000:USD    none         99.00         0.5 → 0.5      -71.2  -0.72     -2.9      [-3.0, -2.9]    -76.9      5.7      <0%     -64.6    -67.8    -71.2    -74.2    -76.9
hantec:express:2000:USD     same_plan    39.00       0.25 → 0.25      -36.9  -0.95     -3.0      [-3.1, -3.0]    -78.9      5.5      <0%     -68.4    -70.9    -73.4    -76.0    -78.9
ftmo:2step:10000:USD        same_plan    97.90       0.5/0.5 → 1      -42.4  -0.43     -3.2      [-3.5, -2.9]   -100.2     22.7      <0%     -43.3    -60.1    -77.4    -91.9   -105.7
hantec:enhanced:10000:USD   same_plan    99.00       0.5/0.5 → 1      -41.7  -0.42     -3.2      [-3.5, -3.0]   -100.4     21.9      <0%     -43.0    -61.0    -78.5    -96.4   -110.4
hantec:enhancedx:10000:USD  none         99.00     0.5/0.5 → 0.5      -94.2  -0.95     -3.9      [-3.9, -3.8]    -96.3      2.1      <0%     -91.8    -93.1    -94.2    -95.1    -96.3
hantec:express:5000:USD     same_plan    59.00       0.25 → 0.25      -53.7  -0.91     -4.4      [-4.5, -4.4]   -116.1      9.3      <0%     -98.2   -102.6   -106.8   -111.3   -116.1
hantec:enhancedx:10000:USD  same_plan    99.00   0.25/0.25 → 0.5      -97.3  -0.98     -4.9      [-4.9, -4.8]   -123.5      6.0      <0%    -112.5   -115.2   -117.5   -120.3   -123.5
hantec:express:10000:USD    same_plan    99.00       0.25 → 0.25      -88.4  -0.89     -7.3      [-7.4, -7.2]   -192.2     16.4      <0%    -160.4   -168.3   -175.8   -183.8   -192.2
```

(Salida real del 2026-10-05 sobre la **raíz de prueba**: la cartera de cuatro estrategias del pool de prueba del Nikkei, umbrales relajados, y decisiones tuyas inventadas para poder probar. **No es un resultado.**)

Primera tabla, la economía: **recompra**, la política de esa fila; **precio** en dólares; **riesgo** por operación en cada fase y en la fondeada (el mejor de la rejilla); **VE/compra**, **VE/$** (por dólar gastado en cuotas) y **VE/mes** con su intervalo al 90 % — ese intervalo es sólo el error de simular, no la duda sobre tu edge; **mono** y **ventaja** (tu VE menos el del mono); **h equil.**, el recorte con el que la caja deja de ganar — «80 %» quiere decir que aguantas perder el 80 % del edge; `>100 %` que gana incluso sin edge; y el VE con cada recorte. La columna `h=100%` es el mono **al riesgo de esa fila**; la columna «mono» es el mono a **su** mejor riesgo, así que `h=100%` nunca la supera.

Segunda tabla, las probabilidades: de aprobar cada fase, de llegar a cobrar alguna vez, **cuotas $** (lo que esperas pagar en total, recompras incluidas), compras totales y compras hasta el primer cobro, cobros por cuenta fondeada y días que vive, y **el desembolso antes del primer cobro** (mediana / p90 / p99): el drawdown de tu caja. «Tope caja» es el % de simulaciones en que el presupuesto se agotó. La última columna es la regla que más cuentas rompe: la que hay que atacar.

Un VE positivo con `h = 0` y negativo con el `h` que has elegido no es una decisión: la frase final nombra esos planes.

Las dos políticas, lado a lado. Un bloque aparte pone cada plan con sus dos VE y su error típico (e.t.: cuánto cambiaría el VE con otros 10.000 caminos), la diferencia, y si esa diferencia cabe dentro del error — si cabe, las dos políticas no se distinguen y no hay nada que elegir:

```
recompra, lado a lado          VE same_plan ± e.t.          VE none ± e.t.  diferencia  ¿dentro del error?
ftmo:2step:10000:USD                   -77.4 ± 3.8              13.6 ± 4.2       -91.0  no
ftmo:1step:10000:USD                   -56.6 ± 3.4              13.1 ± 2.7       -69.8  no
hantec:enhanced:10000:USD              -78.5 ± 4.0               3.8 ± 3.9       -82.3  no
hantec:enhanced:5000:USD               -56.7 ± 0.9              -7.6 ± 2.0       -49.1  no
hantec:endurance:5000:USD              -33.8 ± 0.4             -19.1 ± 0.6       -14.6  no
hantec:express:2000:USD                -73.4 ± 0.4             -33.4 ± 0.2       -39.9  no
hantec:endurance:10000:USD             -68.8 ± 0.8             -39.3 ± 1.2       -29.5  no
hantec:express:5000:USD               -106.8 ± 0.7             -45.1 ± 0.5       -61.7  no
hantec:enhancedx:5000:USD              -70.2 ± 0.3             -56.6 ± 0.4       -13.6  no
hantec:express:10000:USD              -175.8 ± 1.2             -71.2 ± 1.0      -104.6  no
hantec:enhancedx:10000:USD            -117.5 ± 0.6             -94.2 ± 0.7       -23.3  no
```

En esta prueba recomprar es peor en todos los planes, y no por poco. Dos causas: cuando una cuenta ya vale menos de lo que cuesta, comprar más multiplica la pérdida; y las cuentas compradas hacia el final del horizonte pagan la cuota entera y no tienen tiempo de cobrar.

Un ejemplo completo

La primera corrida sobre una cartera de verdad (2026-10-05, **raíz de prueba, no es un resultado**): la cartera K = 4 que el motor eligió para la Hantec Express 10k en el pool de prueba del Nikkei, 10.000 caminos de 504 días, edge centrado en oos1+oos2 (el 20 % del de build), con valores de prueba: 1,10 $/€, sin colchón, 20 % de impuestos, 2 % de comisión, 5 % de cobros denegados, 10 % al año de cierre de la empresa, los dos suelos en `stays`. VE en dólares por banco, con cada recorte:

```
plan                       recompra    precio$     h=0%   h=25%   h=50%   h=75%  h=100%    mono
e.t.  P cobro  cuotas$
ftmo:2step:10000:USD       none          97.90    +43.2   +28.1   +13.6    +1.2   -11.4   -11.4
4.2     13%     97.9
ftmo:2step:10000:USD       same_plan     97.90    -43.3   -60.1   -77.5   -91.9  -105.7  -100.2
3.9     13%    178.7
ftmo:1step:10000:USD       none          86.90    +34.8   +22.8   +13.1    +2.0    -6.7    -6.7
2.7     20%     86.9
hantec:enhanced:10000:USD  none          99.00    +35.0   +19.4    +3.8    -9.0   -20.5   -20.5
3.9     11%     99.0
hantec:express:10000:USD   none          99.00    -64.6   -67.8   -71.2   -74.2   -76.9   -76.9
1.0     13%     99.0
hantec:express:10000:USD   same_plan     99.00   -160.4  -168.3  -175.8  -183.8  -192.2  -192.2
1.2     14%    197.0
```

Lo que enseña. **El plan para el que se eligió la cartera pierde**: la Express 10k deja −71 $ sin recomprar y −176 $ recomprando, y ni con el edge entero (h = 0) sale positiva. Los mismos caminos en una FTMO sin recompra dan +13 $ por 98 $ de cuota — pero esa cartera no se eligió para FTMO, y el capítulo 102 no deja comprarla por eso. Repetido con otra semilla, las tres primeras filas dan +10,3 ± 4,0, +10,6 ± 2,6 y +6,1 ± 3,9: **no se distinguen entre sí**; el orden entre ellas es ruido. Y en todas las filas «none» lo peor que pasa es perder exactamente la cuota.

Sin tus decisiones, se niega:

```
No simulo: faltan decisiones tuyas o datos que el catálogo no trae. Dalos con --set <clave>=
<valor>:
  owner.eurusd — el cambio EUR/USD del día del cálculo (dólares por euro)
  owner.payout_cushion — fracción del tamaño que se deja en la cuenta en cada cobro (0 =
retirar todo)
  owner.tax_rate — fracción de cada cobro que se va en impuestos (0 sólo si tú dices 0)
  owner.payout_fee — fracción de cada cobro que se va en comisiones de pago y divisa
  owner.p_payout_denied — probabilidad de que la empresa deniegue un cobro
  owner.p_firm_closes_year — probabilidad de que la empresa cierre en un año
  gaps.ftmo_1step_floor_after_payout — suelo de pérdida máxima de la fondeada de ftmo 1step
tras un cobro: stays (…) o initial (…)
  gaps.hantec_endurance_payout_days — cada cuántos días naturales cobra la fondeada de hantec
endurance
  gaps.hantec_express_floor_after_payout — suelo de pérdida máxima de la fondeada de hantec
express tras un cobro: stays (…) o initial (…)
```

(El reembolso de la cuota ya no se pregunta: el catálogo lo tiene desde el 2026-10-04 — Hantec no la devuelve, confirmado; FTMO 2-Step se modela sin reembolso hasta que su soporte conteste.)

Qué NO te dice

- **No dice qué comprar**: hace la tabla. La frase la da el capítulo 102.

- **El VE es «sobre el horizonte»**, y el horizonte lo eliges tú al generar los caminos. No es un detalle: con 504 días el trader sin edge pierde en todos los planes; con 1.008 días, tres planes de pérdida máxima fija (FTMO 2-Step, FTMO 1-Step y Hantec Enhanced 10k, sin recompra) le dan un VE **positivo** de 1 a 12 $. No es un fallo: una cuenta fondeada con suelo fijo deja que quien no tiene edge retire, de media, justo el margen de pérdida que le prestan, si le das tiempo. Dos tablas con horizontes distintos no se comparan.
- **El nivel del edge sale de los tramos que tú digas** (`--edge-from`); con pocos días fuera de muestra esa media tiene mucho ruido, y el recorte `h` es la única protección.
- **El intervalo no incluye la duda sobre el edge**, sólo el error de simulación. La duda sobre el edge es la curva VE(h).
- **El mono por defecto no paga costes**: su VE es un techo de lo que vale el plan como lotería.
- **Reglas sin confirmar**: imprime cuáles usa tal como las lee el catálogo, tu lectura del suelo tras el cobro, y dos supuestos propios de la fondeada — que los días mínimos y la consistencia se cuentan desde el cobro anterior, y que el primer cobro llega un ciclo después de empezar la fondeada.
- **El lote mínimo no está**: en cuentas pequeñas el riesgo real va a saltos; aquí es continuo.
- **Add-ons que no cambian la simulación** (noticias, primer cobro a demanda) no se evalúan: sólo suben el precio aquí, aunque puedan cambiar qué estrategias entran en el pool.
- El riesgo elegido es el mejor de la rejilla: si sale siempre el extremo, la rejilla es corta.

Si algo falla

- `No simulo: faltan decisiones tuyas…` — falta una clave de la tabla de arriba; la nombra.
- `No simulo: estos caminos salen de datos reales…` — faltan `--study`, `--symbol`, `--segment`.
- `ModuleNotFoundError: portfolio.common.construct.mc` con `--days` — el generador de caminos de la etapa B aún no está; usa `--paths` con un conjunto ya hecho.

96. Cartera: la búsqueda por rangos — qué estrategias del pool van juntas, elegidas en build

Qué pregunta responde

De un pool de estrategias archivadas, **qué combinación forma la mejor cartera** mirando sólo el tramo `build`. «Mejor» no es una sola cifra: cada combinación admisible se mide con cuatro métricas de su curva diaria y se ordena en cada una; gana la que queda más arriba de media en las cuatro.

Las cuatro métricas, sobre la cartera a pesos iguales (todas las estrategias al mismo riesgo):

métrica	qué mide	mejor
RAR %	la pendiente anual de una recta ajustada a la curva de capital: cuánto crece al año, sin depender de dónde empieza o acaba	alta
R³ (R-cubed)	el RAR dividido por la media de los cinco peores drawdowns y por lo que duraron: crecer mucho con caídas cortas y poco hondas	alta
gain-to-pain	el beneficio medio anual dividido por la media de los N peores drawdowns (N = años del tramo)	alta
P(año perdedor)	qué parte de los años pierde dinero	baja

La puntuación va de 0 a 1: es la media de los cuatro puestos (el de P(año perdedor), al revés). 1 sería ser la mejor en las cuatro a la vez. Decidido el 2026-10-04 por el Council, por delegación tuya (Q10 a).

El mínimo desplazado (decisión tuya del 2026-10-05). Una métrica medida sobre una ventana fija cambia un 10-15 % con sólo mover unos meses dónde empieza. Por eso cada combinación se mide cuatro veces: sobre `build` entero y sobre `build` empezando 1, 2 y 3 meses más tarde, y se queda con lo peor. **Lo peor de qué** lo decidiste el 2026-10-05 (`real.shift_minimum: metric`): cada métrica toma su peor valor de las cuatro ventanas y los puestos se sacan una sola vez, sobre esos valores. Ejemplo: si el RAR de A, B y C es 10, 20, 30 en una ventana y 9, 8, 7 en la otra, los peores valores son 9, 8, 7 y gana **A**. Manda la ventana donde todo va peor. La otra lectura (puestos dentro de cada ventana) no se eligió y ya no existe en el código.

Cuándo lo usas, y cuándo no

Cuando ya hay un pool declarado (capítulo 77) y quieres la cartera para la **cuenta real**. Para una cuenta de fondeo la búsqueda es otra (capítulo 79): allí manda la probabilidad de pasar el plan.

No sirve para decidir si la cartera vale: sólo elige en `build`. El juicio fuera de muestra (`oos1` contra 10.000 carteras al azar, y `oos2` una sola vez con el Sharpe deflactado) es el hito siguiente y **no está construido**. Una cartera salida de aquí no se opera.

Antes de empezar

- El pool declarado y sus estrategias archivadas **con el stop del paso 24** (SL = X·ATR(20)): sin él no hay riesgo fijo y la estrategia se queda fuera, con su nombre en el informe. Hoy las 30 madres archivadas de `Test_USDJPY_donchianUpperCrossUp_H1` no lo llevan; el único pool que lo lleva es el de prueba `Test_nikkei_ref` (capítulo 100).

- No toca SQX. Construir el universo de un pool nuevo usa mucha memoria (capítulo 78): no lo lances con el custodio a pleno rendimiento.
- Ya decidido por ti el 2026-10-05, en `config.yaml`: el mínimo desplazado es el de cada métrica (`real.shift_minimum: metric`), el año perdedor se cuenta por años naturales (`real.losing_year: calendar`), el incremento de una admisión es «finite» (`real.increment`) y, si hace falta el genético, los puestos se toman contra sus carteras de partida (`real.rank_reference: seeds`).

Cómo se ejecuta

```
python3 -m portfolio.common.construct.real --pool desarrollo
python3 -m portfolio.common.construct.real --pool desarrollo --base <carpeta de una run>
```

flag	obligatorio	qué hace
`--pool`	sí	nombre del pool declarado
`--firm`	no	la empresa cuyas prohibiciones se aplican (`AlgoData/portfolio/prohibitions/<empresa>.csv`): las estrategias que esa empresa prohíbe salen del pool **antes** de buscar y se nombran una a una (capítulo 78). Sin el flag no se aplica ninguna
`--set real.shift_months=3`	no	cuántos meses se desplaza el inicio (0 = `build` entero una sola vez)
`--set real.losing_year=rolling`	no	cuenta el año perdedor en todas las ventanas de 252 días en vez de por años naturales
`--base <carpeta de una run>`	no	en vez de buscar, **admite en secuencia** a las estrategias que quedaron fuera de la cartera de esa run (abajo)
`--set pairs.pearson=0.45` (y demás)	no	relaja un filtro de parejas; la salida lo dice en una línea «AJUSTADO» y eso ya no es un resultado

Con más de 100.000 combinaciones admisibles no se prueban todas: busca un algoritmo genético y los puestos se toman contra sus carteras de partida.

La admisión en secuencia (`--base`, decisión tuya del 2026-10-05). Cada candidata entra si ninguna pareja con un miembro incumple los filtros y si sube el Sharpe de la cartera en `build` (Sharpe de las K+1 menos Sharpe de las K). Entra **una por ronda**: la que más lo sube. La cartera crece y las demás se vuelven a juzgar contra la cartera nueva, hasta que ninguna suma. Juzgarlas todas contra la cartera inicial dejaría entrar a dos clones de la misma idea: cada una ayuda sola, juntas no.

Cuántas estrategias caben no se teclea: lo fija la economía del fondeo (decisión del 2026-10-04). Una estrategia cuyo trade más pequeño no llega a 0,01 lotes en una cuenta de 10.000 ni arriesgando el 1 % por operación se queda fuera; con las demás, la cartera puede tener de 1 a tantas como quepan sin que ninguna pareja incumpla los filtros de correlación.

La búsqueda en sí tarda segundos; el universo, lo que diga el capítulo 78.

Qué produce

En `AlgoData/portfolio/runs/<fecha>_<pool>_real/`:

- `search.parquet` — una fila por combinación evaluada: miembros, K, puntuación, las cuatro métricas, el Sharpe diario y el riesgo mínimo por operación.
- `manifest.json` — la cartera elegida, cuántas se probaron, las excluidas y por qué. `verdict` vacío.
- Con `--base`: `admission.csv` dentro de la carpeta de esa run, una fila por juicio y ronda.

Y en el registro (ledger), paso 27, tramo `build`: **cada combinación evaluada cuenta como un intento**, en el estudio del pool y en el de cada estrategia. Ese recuento es el que luego rebaja el Sharpe de la cartera en `oos2`.

Cómo se lee el resultado

Primera run sobre un pool declarado con el stop del paso 24: las 10 estrategias del Nikkei de `Test_nikkei_ref`. **Es una prueba, no un resultado**: corre en la raíz de prueba (capítulo 100) y con los seis filtros de parejas subidos de 0,30 a 0,90, porque las diez son casi clones y a 0,30 ninguna pareja pasa.

```
$ ALGO_DATA_ROOT=…/portfolio/_test/root python3 -m portfolio.common.construct.real --pool Test_nikkei_ref \
      --set pairs.pearson=0.90 … (los seis a 0.90) --set real.shift_minimum=metric
RAÍZ DE PRUEBA: AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
Pool 'Test_nikkei_ref': 10 miembros en el universo, 10 utilizables
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9,
'rolling_recent': 0.9} — no es un resultado
Búsqueda exhaustive: 103 combinaciones admisibles evaluadas en build; rangos tomados entre 103; inicio desplazado 0…3 meses,
mínimo «metric»
CARTERA ELEGIDA: K=1 [c9b701e7] · puntuación 0.871 (0-1, media de los rangos de las 4 métricas) · deja fuera 9 estrategias
  al lote mínimo en 10,000: 0.002% por operación y miembro, 0.002% del balance arriesgado a la vez si coinciden los 1
  RAR 0.016 %/año · R³ 6.63 · gain-to-pain 2.44 · P(año perdedor) 0% · Sharpe diario 0.082
Sin veredicto: es la elección en build; el juicio en oos1 y oos2 es el hito M5.
Carpeta: AlgoData/portfolio/_test/root/portfolio/runs/2026-10-05T0919_Test_nikkei_ref_real
```

- La segunda línea, **«AJUSTADO»**, aparece siempre que un filtro no está en su valor del registro.
- **K=1**: de 103 combinaciones admisibles gana una estrategia sola. Es lo esperable con clones: juntar dos casi iguales no baja los drawdowns y sí baja el RAR (el RAR de una cartera a pesos iguales es la media del de sus miembros: nunca supera al de su mejor miembro).
- La lectura `score` (no elegida, ya retirada) daba la misma cartera y el mismo orden de las 103.
- **«deja fuera N estrategias»** es el coste de la elección, al lado de lo que gana.
- **«arriesgado a la vez»** es K por el riesgo mínimo de cada miembro: lo que la cuenta de 10.000 tiene en juego si todas abren a la vez al lote mínimo. Nadie ha fijado un tope para esa cifra.
- La puntuación **sólo se compara dentro de una misma run** (es un puesto entre las carteras de esa run). Entre runs se comparan las cuatro métricas.

La admisión, leída

Para verla trabajar se le dio como cartera de partida la más floja de las diez (`c29a3efb`):

```
$ ALGO_DATA_ROOT=…/portfolio/_test/root python3 -m portfolio.common.construct.real --pool Test_nikkei_ref \
      --set pairs.…=0.90 --set real.shift_minimum=metric --base <carpeta con la cartera [c29a3efb]>
RAÍZ DE PRUEBA: AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9,
'rolling_recent': 0.9} — no es un resultado
 round candidate  blocked_by  increment  admissible  enters
     1 189a7efb           0   0.010855        True   False
     1 45bc9b7b           0   0.019595        True   False
     1 4806ad30           0   0.021037        True   False
     1 5b9b06ce           0   0.016620        True   False
     1 7f37f07c           0   0.018998        True   False
     1 95656395           0   0.022260        True   False
     1 c9b701e7           0   0.027397        True    True
     1 e1cbf1bc           0   0.022625        True   False
     1 e7550f9c           0   0.021505        True   False
     2 189a7efb           0  -0.000958       False   False
     2 45bc9b7b           1   0.002184       False   False
     2 4806ad30           0   0.003988        True   False
     2 5b9b06ce           0   0.000576        True   False
     2 7f37f07c           1   0.001563       False   False
     2 95656395           0   0.004160        True    True
     2 e1cbf1bc           1   0.003991       False   False
     2 e7550f9c           1   0.003599       False   False
     3 189a7efb           0  -0.001312       False   False
     3 45bc9b7b           2  -0.000019       False   False
     3 4806ad30           0   0.001636        True    True
     3 5b9b06ce           1  -0.001080       False   False
     3 7f37f07c           2  -0.000362       False   False
     3 e1cbf1bc           2   0.001462       False   False
     3 e7550f9c           2   0.001194       False   False
     4 189a7efb           0  -0.001108       False   False
     4 45bc9b7b           2  -0.000340       False   False
     4 5b9b06ce           1  -0.001200       False   False
     4 7f37f07c           2  -0.000642       False   False
     4 e1cbf1bc           2   0.000960       False   False
     4 e7550f9c           2   0.000519       False   False
Entran 3 de 9, en este orden: c9b701e7, 95656395, 4806ad30. Una por ronda: la de mayor incremento «finite» del Sharpe en
build entre las que no veta ninguna pareja; la base crece y las demás se vuelven a juzgar contra ella.
```

- `round` es la ronda; `blocked_by`, con cuántos miembros de la cartera de esa ronda incumple un filtro; `increment`, lo que sumaría al Sharpe diario; `admissible`, si podría entrar en esa ronda; `enters`, la que entra (una por ronda).
- En la ronda 1 las nueve ayudan a la cartera de partida. En secuencia sólo entran tres: una vez dentro las mejores, las demás ya no suman.
- Cada fila es un intento más en el registro.

Un ejemplo completo

La prueba `tests/test_portfolio_ranks.py` lo hace con cuatro estrategias inventadas: A y B ganan lo mismo y sus vaivenes son opuestos; C y D ganan menos. Sólo son admisibles las parejas A-B, A-C y C-D, así que hay 7 carteras posibles (4 sueltas y 3 parejas). Salida real:

```
OK     exhaustive: 4 singles + 3 admissible pairs, no triangle -> 7 combinations
OK     the engine's scores equal the slow independent computation
OK     the best portfolio is the hedged pair {A, B}
       {A,B}: score 0.857 · second best 0.821 · 7 combinaciones
```

Gana A+B porque al juntarlas los vaivenes se anulan: es la primera en R³ y en gain-to-pain, empata en la cabeza en P(año perdedor) y es la segunda en RAR.

`tests/test_portfolio_planted.py` hace la prueba dura, por el mismo camino que el comando y con los filtros a 0,30: diez estrategias inventadas donde la cartera correcta, {A, B}, se conoce de antemano. Un duplicado de A y una cobertura fuerte (correlación −0,64) tienen que quedar fuera, y la prueba comprueba que, si la cobertura se dejara entrar, la búsqueda la elegiría.

Qué NO te dice

- **No dice que la cartera funcione.** Está elegida mirando `build`; la mejor de miles de combinaciones siempre se ve bien en el tramo donde se eligió.
- No compara con carteras al azar: eso se hace sólo fuera de muestra, nunca en `build`.
- Los drawdowns son de la curva diaria (flotante al cierre de cada día incluido), **no del peor momento dentro del día**. Para fondeo, donde manda el flotante intradía, usa el capítulo 79.
- La admisión (`--base`) mira sólo el Sharpe en `build` y los filtros de parejas, no la puntuación por rangos: puede admitir a una estrategia que la búsqueda no habría elegido.
- El orden de entrada depende de la cartera de partida: con otra base entran otras.

Si algo falla

- `real.shift_minimum no tiene valor y no lleva defecto` — alguien le pasó otra lectura con `--set`; la única es `metric`.
- `Ningún miembro utilizable` — ninguna estrategia del pool lleva el stop del paso 24, o a ninguna le llega el lote mínimo.
- `real.rank_reference no tiene valor` — alguien vació ese valor de `config.yaml`; es `seeds`.

100. Cartera · la raíz de prueba — probar el motor de carteras sin tocar los datos de verdad

Qué pregunta responde

¿Cómo se prueba el motor de carteras entero —el que escribe en el ledger, en el archivo y en los pools— sin que una prueba deje una sola fila en los datos reales? Con una **raíz de prueba**: una carpeta aparte que hace de `AlgoData` para ese comando. El código que corre es el de verdad; lo único que cambia es dónde escribe.

Cuándo lo usas, y cuándo no

Úsalo para comprobar que el cableado funciona: que un pool se declara, que el universo concilia, que una búsqueda termina. **No** lo uses para decidir nada: un número que sale de la raíz de prueba no es un resultado. Lleva umbrales relajados a mano, un ledger vacío y, para las estrategias con stop, una curva diaria que no es la de SQX (abajo).

Antes de empezar

Nada. No toca SQX ni el demonio. Para el segundo comando tiene que existir la cosecha real del proyecto (`AlgoData/harvest/<proyecto>/<databank>/`) y el lote del paso 24 con sus `.sqx` (`AlgoData/atrCalculator/<proyecto>/`).

Cómo se ejecuta

Primero se crea la raíz (una vez; repetirlo no rompe nada):

```
python3 -m portfolio.testroot.build
python3 -m portfolio.testroot.build --root /home/sergioguslw/Desktop/AlgoData/portfolio/_test/otra   # una raíz desechable, con su ledger vacío
```

`--root` crea una segunda raíz en la carpeta que se le diga (se niega si es la raíz real o una que la contenga): sirve para probar lo que solo se puede hacer una vez — abrir el oos2 de un pool — sin gastar la raíz de prueba de siempre. Se borra a mano al acabar.

Después, **cualquier** comando del proyecto corre dentro de ella poniéndole delante la variable:

```
ALGO_DATA_ROOT=/home/sergioguslw/Desktop/AlgoData/portfolio/_test/root python3 -m portfolio.common.construct.universe --pool Test_nikkei_ref
```

Todo proceso que corre así lo dice en su primera línea: `RAÍZ DE PRUEBA: … — lo que salga de aquí no es un resultado`. Si esa línea no sale, el comando está escribiendo en los datos de verdad.

Para llevar a la raíz de prueba las estrategias con stop del paso 24 y poder archivarlas:

```
ALGO_DATA_ROOT=/home/sergioguslw/Desktop/AlgoData/portfolio/_test/root python3 -m
portfolio.testroot.harvest --project Test_NIKKEI225_M30_simpleL_ref --databank Stop_p90 --
family simple:long
```

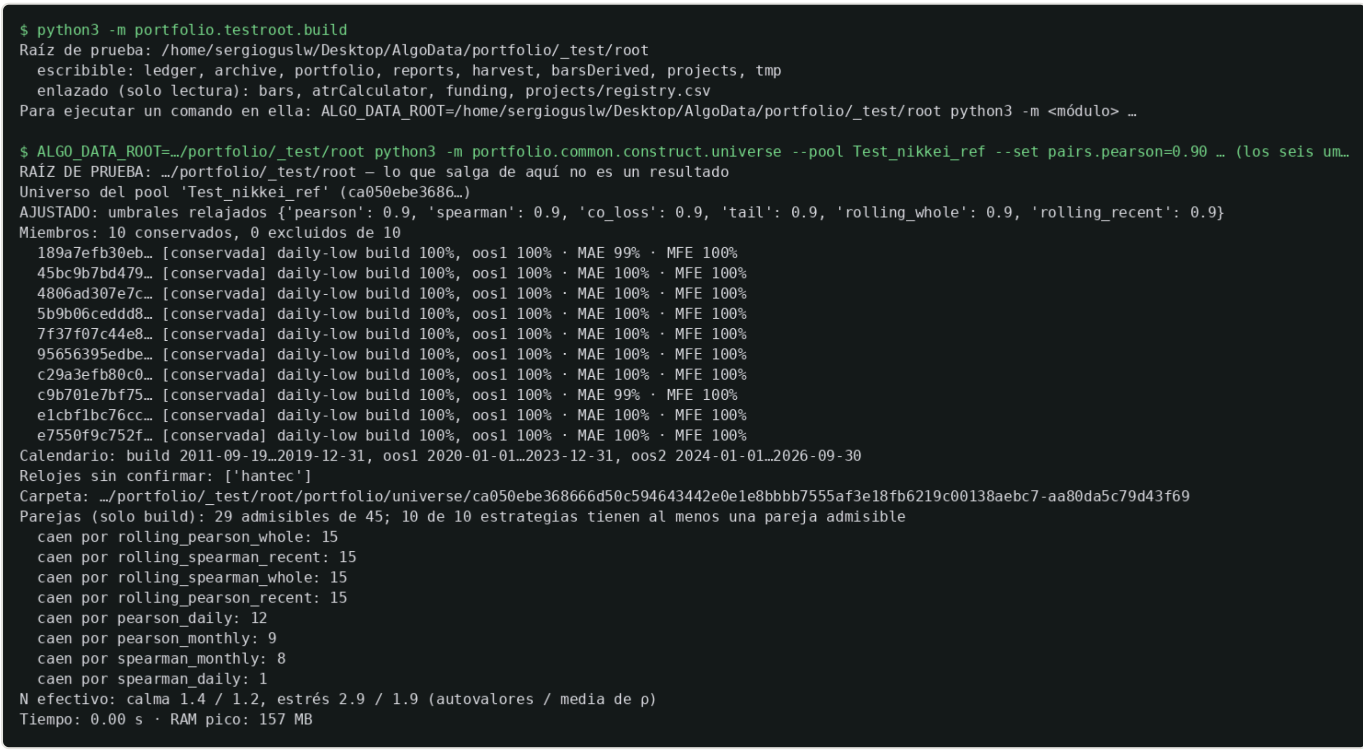
flag	obligatorio	qué hace
`--project`	sí	el proyecto cuya cosecha real se copia
`--databank`	sí	el databank de esa cosecha (las versiones con stop: `Stop_p90`)
`--family`	sí	la familia de plantilla, tal cual la quiere `core.archive` (`simple:long`)

`build` tarda menos de un segundo; `harvest`, unos 40 segundos para 10 estrategias (lee las barras M1). Ninguno toca SQX. `harvest` se niega a correr fuera de una raíz de prueba.

Qué produce

- `build`: la carpeta `AlgoData/portfolio/_test/root/`, con carpetas propias y vacías para todo lo que una prueba escribe (`ledger`, `archive`, `portfolio`, `reports`, `harvest`, `barsDerived`, `projects`, `tmp`) y **enlaces** a lo que solo lee (`bars`, `atrCalculator`, `funding`, `projects/registry.csv`).
- `harvest`: `…/_test/root/harvest/<proyecto>/<databank>/<hoy>/` con las operaciones, las métricas y una curva diaria, y por pantalla la orden de `core.archive` de cada estrategia (se lanzan con la misma variable delante). No sobrescribe: si la cosecha de hoy ya existe, se niega.

Cómo se lee el resultado



```
$ python3 -m portfolio.testroot.build
Raíz de prueba: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root
  escribible: ledger, archive, portfolio, reports, harvest, barsDerived, projects, tmp
  enlazado (solo lectura): bars, atrCalculator, funding, projects/registry.csv
Para ejecutar un comando en ella: ALGO_DATA_ROOT=/home/sergioguslw/Desktop/AlgoData/portfolio/_test/root python3 -m <módulo> …

$ ALGO_DATA_ROOT=…/portfolio/_test/root python3 -m portfolio.common.construct.universe --pool Test_nikkei_ref --set pairs.pearson=0.90 … (los seis um…
RAÍZ DE PRUEBA: …/portfolio/_test/root — lo que salga de aquí no es un resultado
Universo del pool 'Test_nikkei_ref' (ca050ebe3686…)
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9}
Miembros: 10 conservados, 0 excluidos de 10
  189a7efb30eb… [conservada] daily-low build 100%, oos1 100% · MAE 99% · MFE 100%
  45bc9b7bd479… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  4806ad307e7c… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  5b9b06ceddd8… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  7f37f07c44e8… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  95656395edbe… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  c29a3efb80c0… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  c9b701e7bf75… [conservada] daily-low build 100%, oos1 100% · MAE 99% · MFE 100%
  e1cbf1bc76cc… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
  e7550f9c752f… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
Calendario: build 2011-09-19…2019-12-31, oos1 2020-01-01…2023-12-31, oos2 2024-01-01…2026-09-30
Relojes sin confirmar: ['hantec']
Carpeta: …/portfolio/_test/root/portfolio/universe/ca050ebe368666d50c594643442e0e1e8bbbb7555af3e18fb6219c00138aebc7-aa80da5c79d43f69
Parejas (solo build): 29 admisibles de 45; 10 de 10 estrategias tienen al menos una pareja admisible
  caen por rolling_pearson_whole: 15
  caen por rolling_spearman_recent: 15
  caen por rolling_spearman_whole: 15
  caen por rolling_pearson_recent: 15
  caen por pearson_daily: 12
  caen por pearson_monthly: 9
  caen por spearman_monthly: 8
  caen por spearman_daily: 1
N efectivo: calma 1.4 / 1.2, estrés 2.9 / 1.9 (autovalores / media de ρ)
Tiempo: 0.00 s · RAM pico: 157 MB
```

Arriba, lo que crea `build`. Abajo, el universo del pool de prueba: la línea `RAÍZ DE PRUEBA`, la línea `AJUSTADO` (los umbrales de parejas se pasaron a mano a 0,90: con el 0,30 de verdad ninguna de las 45 parejas es admisible, porque las diez estrategias son casi la misma) y las diez conservadas.

- **MAE / MFE 99-100 %** es una comprobación de verdad: compara el peor y el mejor punto de cada operación reconstruidos minuto a minuto contra las columnas que escribió SQX.
- **daily-low 100 %** aquí **no prueba nada**: se compara contra una curva que fabricó el propio motor (ver «Qué NO te dice»).

Un ejemplo completo

El pool de prueba de hoy (2026-10-05), de principio a fin; todas las órdenes menos la primera llevan `ALGO_DATA_ROOT=/home/sergioguslw/Desktop/AlgoData/portfolio/_test/root` delante:

```
$ python3 -m portfolio.testroot.build
Raíz de prueba: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root
  escribible: ledger, archive, portfolio, reports, harvest, barsDerived, projects, tmp
  enlazado (solo lectura): bars, atrCalculator, funding, projects/registry.csv

$ python3 -m portfolio.testroot.harvest --project Test_NIKKEI225_M30_simpleL_ref --databank
Stop_p90 --family simple:long
AVISO: equity.parquet NO es la curva de SQX: es la reconstrucción M1 del propio motor (…)
python3 -m core.archive archive --project Test_NIKKEI225_M30_simpleL_ref --databank Stop_p90 --
identity c29a3efb80c0…34d419c9 --step 24 --family simple:long --sqx …/pass2/sqx/S00V014.sqx   #
Strategy 10.17.411
… (una línea por estrategia: diez)

$ (las diez órdenes de archivo, una a una)

$ python3 -m portfolio.common.construct.inputs.pool declare Test_nikkei_ref --added-by "fase 0
(prueba)" --file pool_ids.txt --note "paso 24, stop p90; curva diaria suplente"
Pool 'Test_nikkei_ref' declarado: 10 miembros, hash ca050ebe3686… →
…/_test/root/portfolio/pools/Test_nikkei_ref.csv

$ python3 -m portfolio.common.construct.universe --pool Test_nikkei_ref --set
pairs.pearson=0.90 --set pairs.spearman=0.90 --set pairs.co_loss=0.90 --set pairs.tail=0.90 --
set pairs.rolling_whole=0.90 --set pairs.rolling_recent=0.90
Miembros: 10 conservados, 0 excluidos de 10
Parejas (solo build): 29 admisibles de 45; 10 de 10 estrategias tienen al menos una pareja
admisible
```

Con los seis umbrales a 0,90 el mayor grupo en el que todas son admisibles entre sí tiene 5 estrategias; a 0,85 tiene 4 (20 parejas), a 0,95 tiene 8.

Qué NO te dice

- **Nada sobre las estrategias.** Es una prueba de cableado con umbrales que tú autorizaste bajar solo para esto.
- **La curva diaria de las versiones con stop no es la de SQX.** El paso 24 no guarda la curva diaria de la estrategia con el stop puesto, así que `harvest` escribe en su lugar la que reconstruye el propio motor. Por eso el «daily-low 100 %» de estas diez es circular. Con los archivos que hay hoy **no se**

puede archivar de verdad una versión con stop: hace falta que el paso 24 guarde esa curva.
- **Los conteos del ledger son los de la prueba.** El ledger de la raíz empieza vacío: el número de estrategias probadas que deflacta un Sharpe allí es el de la prueba, no el del proyecto.
- **No hay oos2.** El pool de prueba no tiene operaciones después de 2023. Para ejercitar el código que lee un segmento posterior se añade `--set calendar.pseudo_oos2_from=2022-01-01`, que llama oos2 a la cola de oos1. No es una reserva ni vale como tal.
- **Los enlaces no protegen.** Lo enlazado (`bars`, `funding`…) es de solo lectura por costumbre: un comando que escribiera ahí escribiría en los datos de verdad. Ninguno del motor de carteras lo hace.

Si algo falla

- `esto solo se ejecuta en una raíz de prueba: ALGO_DATA_ROOT=…`: lanzaste `harvest` sin la variable.
- `la cosecha de prueba de hoy ya existe, bórrala para rehacerla: …`: ya se hizo hoy.
- `el pool 'Test_nikkei_ref' ya está declarado`: un pool queda congelado; otro pool es otro nombre.
- Para empezar de cero: `chmod -R u+w …/_test/root/archive` (lo archivado queda sellado) y borrar `archive`, `harvest`, `portfolio` y `ledger` dentro de `…/_test/root/`; luego `build` otra vez.

101. Cartera · el veredicto de la cuenta real — ¿le gana la cartera elegida al azar, y es real su Sharpe?

Qué pregunta responde

La búsqueda del capítulo 96 elige una cartera mirando solo `build`. Este comando la juzga fuera de muestra y termina en una decisión: **PASA**, **NO PASA** o **NO SE PUEDE JUZGAR**. Son dos pruebas, en este orden, y nunca mezcladas (decisión del Council del 2026-10-04, Q11 d, y tuya del 2026-10-05):

1. **En oos1, contra el azar.** Se sacan 10.000 carteras al azar del mismo pool, **del mismo tamaño** que la tuya y que cumplen los mismos filtros de correlación. La tuya tiene que quedar por encima del 95 % de ellas.
2. **En oos2, una sola vez.** Solo si pasó la primera. El Sharpe de tu cartera en oos2 tiene que seguir siendo positivo después de descontarle la suerte de todo lo que se ha probado en el proyecto entero.

Es el hito M5 de `portfolio/PLAN.md` §5.5.

Cuándo lo usas, y cuándo no

Cuando ya tienes un pool declarado (capítulo 77) y quieres saber si la cartera que sale de él se opera. **No** lo uses para «probar a ver»: la segunda prueba abre oos2, y oos2 **se lee una vez por pool y nunca más**. Si el pool es de prueba, corre en la raíz de prueba (capítulo 100).

No sirve para el fondeo: la cartera de un plan de fondeo se juzga con la etapa B (capítulo 87). Y **no** puede juzgar un pool pequeño: si hay menos de 20 carteras admisibles del tamaño de la tuya, el azar la repite demasiadas veces y el comando lo dice en vez de dar un número.

Antes de empezar

- El pool declarado y sus estrategias en el archivo (capítulos 77 y 78).
- Nada de SQX: no lo toca, se puede lanzar con la GUI abierta.
- Para abrir oos2 hace falta que la puerta del registro lo permita para cada activo de la cartera. En una sesión desatendida no lo permite hasta que tú lo autorices.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.report --pool Test_nikkei_ref
```

flag	obligatorio	qué hace
`--pool`	sí	nombre del pool declarado
`--firm`	no	la empresa cuyas prohibiciones se quitan del pool antes de buscar (capítulo 78)
`--run <carpeta>`	no	juzga la cartera de una búsqueda ya hecha (capítulo 96) **sin buscar otra vez**. Sin el flag, busca y juzga
`--set pairs.pearson=0.45` (y demás)	no	relaja un filtro; la salida dice «AJUSTADO» y ya no es un resultado
`--set real.null_draws=…`, `--set real.null_percentile=…`	no	cambia las 10.000 extracciones o el percentil 95; también marca la salida como «AJUSTADO»

Tarda segundos: 3,6 s con el universo ya calculado, unos 12 s si hay que buscar primero. Las 10.000 carteras al azar de 5 estrategias sobre un pool de 50 cuestan unos 10 s.

Qué produce

En la carpeta de la run, `AlgoData/portfolio/runs/<fecha>_<pool>_real/`:

archivo	qué es
`verdict.json`	el veredicto entero: la cartera, la lectura de oos1, la de oos2, la diversificación y el coste
`manifest.json`	se le añade el veredicto (sobrescribe el campo `verdict`, que la búsqueda deja vacío)
`cartera.html`, `cartera.json`, `cartera.md`	el informe para leer: Veredicto, Universo, Parejas, Búsqueda y Cartera
`handoff/portfolio.json`	la cartera para quien venga después: miembros con su versión, feed y reloj, pesos, el veredicto, el N del registro
`handoff/daily.parquet`, `handoff/trades.parquet`	el P&L diario y las operaciones de cada miembro

Y en el registro, paso 27: **una fila por cada tramo que el veredicto mira** (oos1 siempre; oos2 solo si se abre), en el estudio del pool y en el de cada estrategia. La fila de oos2 guarda la lectura completa: es lo que impide leerlo dos veces.

Cómo se lee el resultado

```
$ python3 -m portfolio.common.construct.report --pool Test_nikkei_ref --run <run K=3> [--set …]
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9, 'pseudo_oos2…
La cartera de K=3 [4806ad30, 95656395, c9b701e7] (de 10 utilizables; 103 combinaciones probadas en build; deja fuera 7):
  oos1 (2020-01-02…2021-12-31) contra 10,000 carteras admisibles al azar de K=3 (37 combinaciones admisibles distintas de ese tamaño; 37 salieron):
    puntuación por rangos 0.815 frente al percentil 95 del azar 0.822 (mediana 0.499); la elegida queda en el percentil 91.6% → NO PASA
    en oos1: RAR 3.9272 por unidad de riesgo y año · R³ 3.92 · gain-to-pain 0.68 · P(año perdedor) 0% · Sharpe diario 0.0426
  oos2: no se lee — oos1 no se ha pasado: oos2 no se abre
  diversificación en oos1: 1 de 3 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: NO PASA (oos1) — esta combinación no se opera.
Carpeta: <raíz de prueba>/portfolio/runs/2026-10-05T0954_Test_nikkei_ref_real_K3_forzada (verdict.json, cartera.html, handoff/)
```

Línea a línea:

- **«contra 10 000 carteras admisibles al azar de K=3 (37 combinaciones admisibles distintas…)»**: cuántas carteras distintas de ese tamaño existen en el pool. Con 37, las 10.000 extracciones repiten cada una muchas veces; el número que importa es el 37.
- **«puntuación por rangos 0.815 frente al percentil 95 del azar 0.822»**: la puntuación es la misma de la búsqueda (capítulo 96) — el puesto medio de la cartera en cuatro métricas: RAR, R³, gain-to-pain y P(año perdedor) — pero ahora medida en oos1 y con los puestos tomados entre las 10.000 al azar y la tuya. Va de 0 a 1; 0,5 es la mitad de la tabla. **Para pasar hay que superar el percentil 95**, no la mediana. Aquí 0,815 < 0,822: no pasa, por poco.
- **«en oos1: RAR … Sharpe diario …»**: las cifras de tu cartera en oos1, para que las veas; no deciden.
- **«oos2: no se lee»**: no pasó oos1, así que oos2 sigue sin abrir. Eso es bueno: no se ha gastado.
- **«diversificación en oos1»**: las correlaciones de las parejas de la cartera medidas otra vez fuera de muestra. Se informa; no decide.
- **«Veredicto: …»**: la decisión.

Cuando oos1 se pasa, la línea de oos2 dice el Sharpe diario de la cartera, **lo que el azar da como mejor de N intentos** (N es el recuento de todo lo probado en el proyecto) y si el primero supera al segundo.

Si el pool es demasiado pequeño:

```
$ python3 -m portfolio.common.construct.report --pool Test_nikkei_ref [--set …]
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9, 'pseudo_oos2…
La cartera de K=1 [c9b701e7] (de 10 utilizables; 103 combinaciones probadas en build; deja fuera 9):
  oos1 (2020-01-02…2021-12-31) contra 10,000 carteras admisibles al azar de K=1 (10 combinaciones admisibles distintas de ese tamaño; 10 salieron):
    NO DISCRIMINA: la cartera elegida es ella sola el 9.8% de las extracciones, más que el 5% que deja libre el percentil 95; el listón sería su prop…
    en oos1: RAR 5.6309 por unidad de riesgo y año · R³ 6.07 · gain-to-pain 1.14 · P(año perdedor) 0% · Sharpe diario 0.0616
  oos2: no se lee — oos1 no se ha pasado: oos2 no se abre
  diversificación en oos1: 0 de 0 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: NO SE PUEDE JUZGAR (el azar repite la cartera elegida) — no se aprueba ni se descarta nada.
Carpeta: <raíz de prueba>/portfolio/runs/2026-10-05T0954_Test_nikkei_ref_real (verdict.json, cartera.html, handoff/)
```

Con 10 estrategias y K=1 solo hay 10 carteras posibles: la tuya sale el 9,8 % de las veces, más del 5 % que deja libre el percentil 95. El listón sería su propia cifra. **No es un suspenso ni un aprobado**: no se puede juzgar con este pool.

Un ejemplo completo

Sobre `Test_nikkei_ref` en la raíz de prueba (capítulo 100), con los filtros relajados a 0,90 y un oos2 de mentira que empieza en 2022 — **no es un resultado**:

```
export ALGO_DATA_ROOT=~/Desktop/AlgoData/portfolio/_test/root
python3 -m portfolio.common.construct.report --pool Test_nikkei_ref \
    --set pairs.pearson=0.90 --set pairs.spearman=0.90 --set pairs.co_loss=0.90 \
    --set pairs.tail=0.90 --set pairs.rolling_whole=0.90 --set pairs.rolling_recent=0.90 \
    --set calendar.pseudo_oos2_from=2022-01-01
```

```
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9, 'pseudo_oos2_from': '2022-01-01'} — no es un resultado
La cartera de K=1 [c9b701e7] (de 10 utilizables; 103 combinaciones probadas en build; deja fuera 9):
  oos1 (2020-01-02…2021-12-31) contra 10 000 carteras admisibles al azar de K=1 (10 combinaciones admisibles distintas de ese tamaño; 10 salieron):
    NO DISCRIMINA: la cartera elegida es ella sola el 9.8% de las extracciones, más que el 5% que deja libre el percentil 95; el listón sería su propia cifra y el percentil no se lee. Hace falta un pool con más combinaciones admisibles.
    en oos1: RAR 5.6309 por unidad de riesgo y año · R³ 6.07 · gain-to-pain 1.14 · P(año perdedor) 0% · Sharpe diario 0.0616
  oos2: no se lee — oos1 no se ha pasado: oos2 no se abre
  diversificación en oos1: 0 de 0 parejas pasan de algún umbral, 0 sin el solape mínimo para juzgarlas (se informa, no decide)
Veredicto: NO SE PUEDE JUZGAR (el azar repite la cartera elegida) — no se aprueba ni se descarta nada.
```

La búsqueda eligió una sola estrategia (las diez son casi clones), y con una sola no hay azar contra el que medirla. La captura de arriba con K=3 es el mismo pool con un trío puesto a mano para ver la prueba funcionar: 37 tríos admisibles, percentil 91,6 %, no pasa.

Qué NO te dice

- **No dice que las estrategias tengan ventaja.** Las carteras al azar salen del mismo pool, así que la ventaja del pool está también en el azar. Dice si la búsqueda **eligió mejor que el azar entre las admisibles**.
- **Pasar oos1 no es pasar.** Sin oos2 la última línea es «SIN VEREDICTO», no un aprobado.
- **Un «PASA» no dice cuánto arriesgar** ni que la cartera vaya a seguir funcionando: después de oos2 lo único que la valida es la incubación en vivo.
- **Cada vez que lo lanzas, oos1 se sortea de nuevo** y se apunta una mirada más en el registro. Una cartera que queda justo en el listón puede pasar una vez y no otra: repetir hasta que pase es hacerse trampa.
- **El N que descuenta el Sharpe está inflado**: cada búsqueda de cartera cuenta una vez en el estudio del pool y otra en el de cada estrategia. Va en contra de la cartera, no a favor.

Si algo falla

- «NO SE PUEDE JUZGAR (el azar repite la cartera elegida)»: menos de 20 carteras admisibles de ese tamaño. Hace falta un pool mayor o menos parecido entre sí.
- «SIN VEREDICTO: oos1 pasado, oos2 sin leer (el estudio del pool no guarda Sharpes de build…)»: el pool no ha pasado por la búsqueda del capítulo 96 y no hay con qué descontar el Sharpe. oos2 no se ha abierto.
- «el oos2 de este pool ya lo leyó otra cartera»: oos2 está gastado para ese pool, por esta vía o por la de fondeo. No hay segunda lectura.
- La puerta del registro se niega a abrir oos2: falta tu autorización para ese activo.

102. La decisión del fondeo — qué cuenta comprar, en una frase, con el coste al lado

Qué pregunta responde

Con las tablas de caja ya hechas (capítulo 88): **¿qué challenge compro, a qué riesgo y con qué política de recompra — o ninguno?** Lee esas tablas, pasa cada fila por los filtros de abajo y escribe una sola frase: «compra esto; CUESTA tanto; GANA tanto», o «ninguno», con el motivo y la fila que más cerca se quedó. No simula nada: decide con lo que la caja ya calculó.

Los filtros, en orden

Una fila (un plan con una política de recompra) se puede comprar sólo si pasa los seis:

1. **La cartera es la de ese plan.** El motor elige una cartera por plan; una tabla simulada con la cartera de otra cuenta se enseña, pero no se compra con ella.
2. **La cartera tiene veredicto favorable** (capítulo 87). «Sin criterio» o «sin veredicto» no es favorable.
3. **El edge fuera de muestra es positivo.** Si no lo es, no hay nada que recortar.
4. **El desembolso antes del primer cobro cabe en tu caja** en el 90 % de los casos.
5. **El VE_banco con el recorte que decide (h = 50 %) es positivo.**
6. **Y es mayor que el del mono**: la cuenta gana por tus estrategias y no por ser una lotería.

De las que pasan, elige la de mayor VE. Si la segunda queda a menos de dos errores típicos de la primera lo dice: es un empate, y elegir entre ellas es elegir ruido.

Cuándo lo usas, y cuándo no

Después del capítulo 88, cuando quieras la respuesta y no la tabla. **No** sirve para comparar tablas hechas con decisiones tuyas distintas (otro cambio, otros impuestos, otro colchón): se niega. Tampoco sirve con las ofertas del día: decide a precio de lista; lo que mueve una oferta lo dice `python3 -m portfolio.funded.deals.worth` (capítulo 74).

Antes de empezar

No toca SQX ni MetaTrader. Tiene que existir al menos una tabla de caja en `AlgoData/portfolio/bank/`, hecha con caminos de una cartera real (las sintéticas no cuentan) y con los recortes y la recompra que tienes fijados hoy.

Cómo se ejecuta

```
python3 -m portfolio.funded.decide.run
```

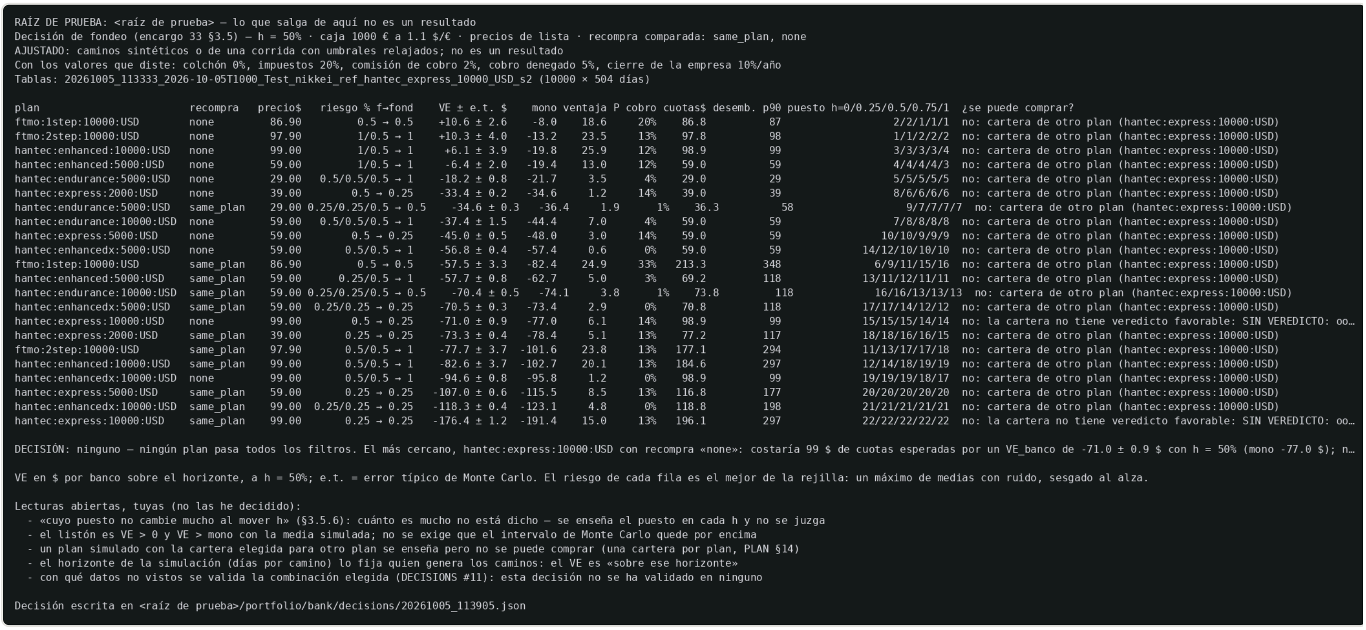
flag	obligatorio	qué hace
`--bank <carpeta>`	no	usar sólo esa tabla de caja; se puede repetir. Sin él, usa todas las de `AlgoData/portfolio/bank/`, y de cada plan la más reciente hecha con su propia cartera
`--ignore-verdict`	no	**sólo para probar el programa**: se salta el filtro 2. La salida sale marcada AJUSTADO y no es una decisión
`--out`	no	dónde escribir el `.json` de la decisión

Tarda menos de un segundo.

Qué produce

La tabla y la frase en pantalla, y `AlgoData/portfolio/bank/decisions/<fecha>.json` con lo mismo: la frase, cada fila con su motivo, la elegida, la segunda, si hay empate, las decisiones tuyas con las que se hicieron las tablas y las lecturas que quedan abiertas. No sobrescribe nada ni apunta nada en el ledger (las pruebas ya las contó la caja).

Cómo se lee el resultado



```
RAÍZ DE PRUEBA: <raíz de prueba> — lo que salga de aquí no es un resultado
Decisión de fondeo (encargo 33 §3.5) — h = 50% · caja 1000 € a 1.1 $/€ · precios de lista · recompra comparada: same_plan, none
AJUSTADO: caminos sintéticos o de una corrida con umbrales relajados; no es un resultado
Con los valores que diste: colchón 0%, impuestos 20%, comisión de cobro 2%, cobro denegado 5%, cierre de la empresa 10%/año
Tablas: 20261005_113333_2026-10-05T1000_Test_nikkei_ref_hantec_express_10000_USD_s2 (10000 × 504 días)

plan                        recompra   precio$   riesgo % f→fond    VE ± e.t. $    mono ventaja P cobro cuotas$ desemb. p90 puesto h=0/0.25/0.5/0.75/1  ¿se puede comprar?
ftmo:1step:10000:USD        none         86.90         0.5 → 0.5    +10.6 ± 2.6    -8.0    18.6     20%    86.8          87                      2/2/1/1/1  no: cartera de otro plan (hantec:express:10000:USD)
ftmo:2step:10000:USD        none         97.90         1/0.5 → 1    +10.3 ± 4.0   -13.2    23.5     13%    97.8          98                      1/1/2/2/2  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhanced:10000:USD   none         99.00         1/0.5 → 1     +6.1 ± 3.9   -19.8    25.9     12%    98.9          99                      3/3/3/3/4  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhanced:5000:USD    none         59.00         1/0.5 → 1     -6.4 ± 2.0   -19.4    13.0     12%    59.0          59                      4/4/4/4/3  no: cartera de otro plan (hantec:express:10000:USD)
hantec:endurance:5000:USD   none         29.00   0.5/0.5/0.5 → 1    -18.2 ± 0.8   -21.7     3.5      4%    29.0          29                      5/5/5/5/5  no: cartera de otro plan (hantec:express:10000:USD)
hantec:express:2000:USD     none         39.00        0.5 → 0.25    -33.4 ± 0.2   -34.6     1.2     14%    39.0          39                      8/6/6/6/6  no: cartera de otro plan (hantec:express:10000:USD)
hantec:endurance:5000:USD   same_plan    29.00 0.25/0.25/0.5 → 0.5    -34.6 ± 0.3   -36.4     1.9      1%    36.3          58                      9/7/7/7/7  no: cartera de otro plan (hantec:express:10000:USD)
hantec:endurance:10000:USD  none         59.00   0.5/0.5/0.5 → 1    -37.4 ± 1.5   -44.4     7.0      4%    59.0          59                      7/8/8/8/8  no: cartera de otro plan (hantec:express:10000:USD)
hantec:express:5000:USD     none         59.00        0.5 → 0.25    -45.0 ± 0.5   -48.0     3.0     14%    59.0          59                    10/10/9/9/9  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhancedx:5000:USD   none         59.00       0.5/0.5 → 1    -56.8 ± 0.4   -57.4     0.6      0%    59.0          59                 14/12/10/10/10  no: cartera de otro plan (hantec:express:10000:USD)
ftmo:1step:10000:USD        same_plan    86.90         0.5 → 0.5    -57.5 ± 3.3   -82.4    24.9     33%   213.3         348                   6/9/11/15/16  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhanced:5000:USD    same_plan    59.00      0.25/0.5 → 1    -57.7 ± 0.8   -62.7     5.0      3%    69.2         118                 13/11/12/11/11  no: cartera de otro plan (hantec:express:10000:USD)
hantec:endurance:10000:USD  same_plan    59.00 0.25/0.25/0.5 → 0.5    -70.4 ± 0.5   -74.1     3.8      1%    73.8         118                 16/16/13/13/13  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhancedx:5000:USD   same_plan    59.00  0.25/0.25 → 0.25    -70.5 ± 0.3   -73.4     2.9      0%    70.8         118                 17/17/14/12/12  no: cartera de otro plan (hantec:express:10000:USD)
hantec:express:10000:USD    none         99.00        0.5 → 0.25    -71.0 ± 0.9   -77.0     6.1     14%    98.9          99                 15/15/15/14/14  no: la cartera no tiene veredicto favorable: SIN VEREDICTO: oo…
hantec:express:2000:USD     same_plan    39.00       0.25 → 0.25    -73.3 ± 0.4   -78.4     5.1     13%    77.2         117                 18/18/16/16/15  no: cartera de otro plan (hantec:express:10000:USD)
ftmo:2step:10000:USD        same_plan    97.90       0.5/0.5 → 1    -77.7 ± 3.7  -101.6    23.8     13%   177.1         294                 11/13/17/17/18  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhanced:10000:USD   same_plan    99.00       0.5/0.5 → 1    -82.6 ± 3.7  -102.7    20.1     13%   184.6         297                 12/14/18/19/19  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhancedx:10000:USD  none         99.00       0.5/0.5 → 1    -94.6 ± 0.8   -95.8     1.2      0%    98.9          99                 19/19/19/18/17  no: cartera de otro plan (hantec:express:10000:USD)
hantec:express:5000:USD     same_plan    59.00       0.25 → 0.25   -107.0 ± 0.6  -115.5     8.5     13%   116.8         177                 20/20/20/20/20  no: cartera de otro plan (hantec:express:10000:USD)
hantec:enhancedx:10000:USD  same_plan    99.00  0.25/0.25 → 0.25   -118.3 ± 0.4  -123.1     4.8      0%   118.8         198                 21/21/21/21/21  no: cartera de otro plan (hantec:express:10000:USD)
hantec:express:10000:USD    same_plan    99.00       0.25 → 0.25   -176.4 ± 1.2  -191.4    15.0     13%   196.1         297                 22/22/22/22/22  no: la cartera no tiene veredicto favorable: SIN VEREDICTO: oo…

DECISIÓN: ninguno — ningún plan pasa todos los filtros. El más cercano, hantec:express:10000:USD con recompra «none»: costaría 99 $ de cuotas esperadas por un VE_banco de -71.0 ± 0.9 $ con h = 50% (mono -77.0 $); n…

VE en $ por banco sobre el horizonte, a h = 50%; e.t. = error típico de Monte Carlo. El riesgo de cada fila es el mejor de la rejilla: un máximo de medias con ruido, sesgado al alza.

Lecturas abiertas, tuyas (no las he decidido):
  - «cuyo puesto no cambie mucho al mover h» (§3.5.6): cuánto es mucho no está dicho — se enseña el puesto en cada h y no se juzga
  - el listón es VE > 0 y VE > mono con la media simulada; no se exige que el intervalo de Monte Carlo quede por encima
  - un plan simulado con la cartera elegida para otro plan se enseña pero no se puede comprar (una cartera por plan, PLAN §14)
  - el horizonte de la simulación (días por camino) lo fija quien genera los caminos: el VE es «sobre ese horizonte»
  - con qué datos no vistos se valida la combinación elegida (DECISIONS #11): esta decisión no se ha validado en ninguno

Decisión escrita en <raíz de prueba>/portfolio/bank/decisions/20261005_113905.json
```

(Salida real del 2026-10-05 sobre la **raíz de prueba**: cartera del pool de prueba del Nikkei, umbrales relajados y decisiones tuyas inventadas para probar. **No es un resultado.**)

- **VE ± e.t.**: el valor esperado de tu banco con h = 50 % sobre el horizonte simulado, y su error típico: cuánto cambiaría con otros 10.000 caminos. Dos filas cuyos VE distan menos de dos errores no se distinguen.
- **mono** y **ventaja**: lo que ganaría la misma cartera sin edge, y cuánto la superas.
- **P cobro**: probabilidad de llegar a cobrar alguna vez. **cuotas$**: lo que esperas pagar en total. **desemb. p90**: lo que tienes que adelantar antes del primer cobro en el 90 % de los casos.
- **puesto h=0/0.25/0.5/0.75/1**: el puesto de la fila entre todas con cada recorte. Una fila que es 1ª con h = 0 y 15ª con h = 0,75 depende de que el edge se mantenga entero.
- **¿se puede comprar?**: «sí», o el primer filtro que no pasa.

Un ejemplo completo

La frase de la captura, entera:

```
DECISIÓN: ninguno — ningún plan pasa todos los filtros. El más cercano,
hantec:express:10000:USD con recompra «none»: costaría 99 $ de cuotas esperadas por un VE_banco
de -71.0 ± 0.9 $ con h = 50% (mono -77.0 $); no se compra porque la cartera no tiene veredicto
favorable: SIN VEREDICTO: oos1 pasado, oos2 sin leer (…)
```

Se lee: la única cartera propia es la de la Hantec Express 10k, y no tiene veredicto favorable; y aunque lo tuviera, su caja pierde 71 $ por cada 99 $ de cuota. Las FTMO salen arriba con +10 $, pero con una cartera que no se eligió para ellas: no se compran. La respuesta es «ninguno».

Cuando las tablas no son comparables, se niega:

```
No decido: las tablas de caja se simularon con valores distintos (owner.* o gaps.*) y sus VE no
se pueden comparar. Vuelve a simularlas con los mismos, o pásalas con --bank.
```

Qué NO te dice

Lo imprime al final de cada salida, bajo «Lecturas abiertas, tuyas»:

- **Cuánto puede moverse el puesto al cambiar `h`** antes de llamar frágil a una elección: enseña el puesto con cada recorte y no lo juzga.
- **Si el listón debe ser el VE o el extremo bajo de su intervalo**: hoy exige VE > 0 y VE > mono con la media simulada.
- **El horizonte**: el VE es sobre los días que duran los caminos, y lo eliges al generarlos. Con horizontes largos el mono gana en algunos planes (capítulo 88).
- **Que la combinación esté validada en datos no vistos**: no lo está en ninguno.
- **El mono de estas tablas no paga costes** salvo que la caja se hiciera con `--monkey`.

Si algo falla

- `No decido: no hay ninguna tabla de caja sobre caminos reales…` — corre antes el capítulo 88.
- `No decido: las tablas de caja se simularon con valores distintos…` — rehaz la más antigua con los valores de hoy, o pasa sólo una con `--bank`.
- `No decido: la tabla no se simuló con tus decisiones vigentes…` — la tabla es anterior a un cambio de `owner.haircuts`, `owner.haircut_decides` u `owner.rebuy`; la frase dice cuál. Rehazla.

103. Cartera · el walk-forward de pesos — ¿algún método de pesos le gana a pesos iguales fuera de muestra?

Qué pregunta responde

La cartera reparte el riesgo a partes iguales entre sus estrategias, y los pesos **no se tocan nunca** (decisión #5 del Council, 2026-10-04). El Council dejó escrita la única cosa que revertiría esa decisión: *un método de pesos que solo le gane a pesos iguales cuando se re-estima*. Este comando lo comprueba. Para cada método responde dos cosas:

1. **Con los pesos fijos** — estimados una vez en `build` y mantenidos —, ¿le gana a pesos iguales en `oos1`?
2. **Re-estimando** los pesos cada cierto tiempo, ¿le gana? ¿Y solo entonces?

Es un estudio que **informa y no cambia nada**: la búsqueda y el veredicto siguen usando pesos iguales. Es el hito M8 de `portfolio/PLAN.md` §5.4.

Los métodos que compara son los que nombra el plan:

método	qué hace, en cristiano
`equal`	a cada estrategia lo mismo. La referencia
`risk_parity`	más peso a la que menos se mueve: cada una pesa 1 / su volatilidad diaria
`min_variance`	los pesos que dan la cartera más tranquila posible, mirando también cómo se mueven juntas. La matriz de covarianzas se «encoge» (Ledoit-Wolf) para no ajustarse al ruido
`hrp`	agrupa las estrategias parecidas en un árbol y reparte el riesgo rama a rama (Hierarchical Risk Parity)

El «handcrafting» de Carver, que el plan también nombra, **no está construido**: necesita umbrales de agrupación que nadie ha decidido.

Cuándo lo usas, y cuándo no

Cuando ya tienes una cartera elegida (capítulos 96 y 101) y quieres saber si merece la pena dejar los pesos iguales. **No** lo uses para elegir un método a ojo: cada vez que lo corres es una mirada más a `oos1` y queda apuntada en el registro. **No** sirve con una cartera de una sola estrategia (no hay pesos que repartir). Y **nunca abre `oos2`**.

Antes de empezar

- El pool declarado y la cartera elegida: la carpeta de una run (capítulo 96 o 101), o las identidades escritas a mano.
- Nada de SQX: no lo toca.
- El listón `wf.confidence` no tiene valor por defecto: es tuyo. Sin él el comando se niega.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.wf --pool Test_nikkei_ref \
    --run ~/Desktop/AlgoData/portfolio/runs/2026-10-05T0954_Test_nikkei_ref_real \
    --set wf.confidence=0.95
```

flag	obligatorio	qué hace
`--pool`	sí	nombre del pool declarado
`--run <carpeta>`	uno de los dos	la cartera elegida de esa run
`--members a1b2,c3d4,…`	uno de los dos	la cartera escrita a mano, cada estrategia por el principio de su identidad
`--firm`	no	la empresa cuyas prohibiciones se quitan del pool
`--set wf.confidence=0.95`	**sí**	el intervalo que una diferencia tiene que superar entera para contar como «gana»
`--set wf.step_years=1`	no	cada cuántos años se re-estiman los pesos, siempre el 1 de enero. Por defecto 3 (el número de AlphaForge)
`--set wf.lookback_years=6`	no	con cuántos años de historia se estiman. Por defecto 0 = con todo lo anterior, de modo que la primera estimación es `build` entero
`--set pairs.pearson=0.45` (y demás)	no	relaja un filtro; la salida dice «AJUSTADO» y ya no es un resultado

Tarda segundos: 4 s con el universo ya calculado, 12 s si hay que calcularlo.

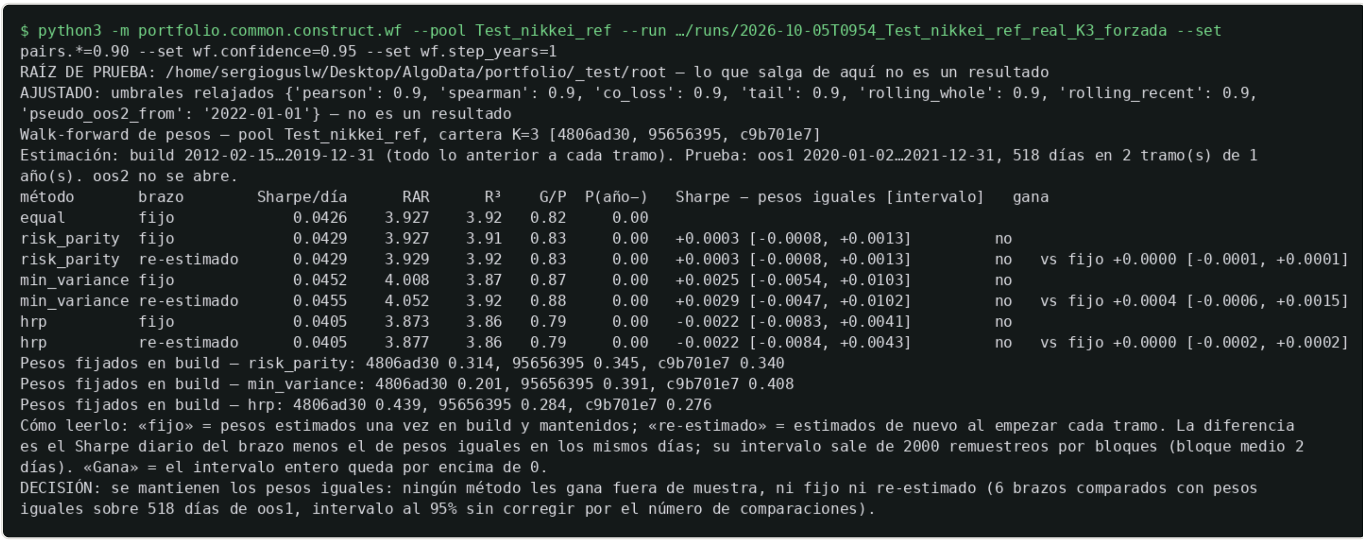
Qué produce

Una carpeta `AlgoData/portfolio/weights_wf/<fecha>_<pool>_K<tamaño>_<huella>/`:

archivo	qué es
`table.parquet`	una fila por método y brazo (fijo / re-estimado): Sharpe, RAR, R^3, gain-to-pain, P(año perdedor), la diferencia con pesos iguales y su intervalo, y si gana
`weights.parquet`	los pesos de cada método en cada tramo
`result.json`	la decisión en una frase, el estado de cada método, los tramos y los parámetros usados

Y en el registro, paso 27, **dos filas** (en el estudio del pool y en el de cada estrategia): una por `build` (donde se estiman los pesos) y otra por `oos1` (donde se comparan), que cuenta cada brazo comparado como una prueba más.

Cómo se lee el resultado



```
$ python3 -m portfolio.common.construct.wf --pool Test_nikkei_ref --run …/runs/2026-10-05T0954_Test_nikkei_ref_real_K3_forzada --set
pairs.*=0.90 --set wf.confidence=0.95 --set wf.step_years=1
RAÍZ DE PRUEBA: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
AJUSTADO: umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9, 'rolling_recent': 0.9,
'pseudo_oos2_from': '2022-01-01'} — no es un resultado
Walk-forward de pesos — pool Test_nikkei_ref, cartera K=3 [4806ad30, 95656395, c9b701e7]
Estimación: build 2012-02-15…2019-12-31 (todo lo anterior a cada tramo). Prueba: oos1 2020-01-02…2021-12-31, 518 días en 2 tramo(s) de 1
año(s). oos2 no se abre.
método       brazo        Sharpe/día      RAR      R³    G/P  P(año−)   Sharpe − pesos iguales [intervalo]   gana
equal        fijo             0.0426    3.927    3.92   0.82     0.00
risk_parity  fijo             0.0429    3.927    3.91   0.83     0.00   +0.0003 [-0.0008, +0.0013]         no
risk_parity  re-estimado      0.0429    3.929    3.92   0.83     0.00   +0.0003 [-0.0008, +0.0013]         no   vs fijo +0.0000 [-0.0001, +0.0001]
min_variance fijo             0.0452    4.008    3.87   0.87     0.00   +0.0025 [-0.0054, +0.0103]         no
min_variance re-estimado      0.0455    4.052    3.92   0.88     0.00   +0.0029 [-0.0047, +0.0102]         no   vs fijo +0.0004 [-0.0006, +0.0015]
hrp          fijo             0.0405    3.873    3.86   0.79     0.00   -0.0022 [-0.0083, +0.0041]         no
hrp          re-estimado      0.0405    3.877    3.86   0.79     0.00   -0.0022 [-0.0084, +0.0043]         no   vs fijo +0.0000 [-0.0002, +0.0002]
Pesos fijados en build — risk_parity: 4806ad30 0.314, 95656395 0.345, c9b701e7 0.340
Pesos fijados en build — min_variance: 4806ad30 0.201, 95656395 0.391, c9b701e7 0.408
Pesos fijados en build — hrp: 4806ad30 0.439, 95656395 0.284, c9b701e7 0.276
Cómo leerlo: «fijo» = pesos estimados una vez en build y mantenidos; «re-estimado» = estimados de nuevo al empezar cada tramo. La diferencia
es el Sharpe diario del brazo menos el de pesos iguales en los mismos días; su intervalo sale de 2000 remuestreos por bloques (bloque medio 2
días). «Gana» = el intervalo entero queda por encima de 0.
DECISIÓN: se mantienen los pesos iguales: ningún método les gana fuera de muestra, ni fijo ni re-estimado (6 brazos comparados con pesos
iguales sobre 518 días de oos1, intervalo al 95% sin corregir por el número de comparaciones).
```

- **«Estimación … Prueba …»**: los pesos se calculan con los días de `build` y se prueban en los de `oos1`. «2 tramo(s) de 1 año(s)»: `oos1` se parte en trozos; al empezar cada uno, el brazo re-estimado recalcula los pesos con todo lo anterior.
- **brazo «fijo»**: los pesos de `build`, sin tocar. Es lo que haría la cartera hoy si usara ese método.
- **brazo «re-estimado»**: los pesos recalculados al empezar cada tramo.
- **Sharpe/día**: ganancia media diaria dividida por lo que oscila. Más alto, mejor. No está anualizado.
- **RAR, R³, G/P, P(año−)**: las cuatro métricas de la búsqueda (capítulo 96), aquí medidas en `oos1`.
- **«Sharpe − pesos iguales [intervalo]»**: cuánto mejor (o peor) es el Sharpe de ese brazo que el de pesos iguales **en los mismos días**, y entre qué dos valores podría estar de verdad esa diferencia. El intervalo sale de volver a barajar los días de `oos1` 2.000 veces, en bloques.
- **«gana»**: «sí» solo si el intervalo **entero** queda por encima de 0. Un +0,0025 con intervalo [−0,0054, +0,0103] no es ganar: el cero está dentro.
- **«vs fijo»**: lo mismo, pero comparando el brazo re-estimado con el mismo método sin re-estimar.
- **«DECISIÓN»**: una de tres. *Se mantienen los pesos iguales* (nadie gana). *X gana con los pesos fijados una vez* (candidato a que tú lo selecciones; no rebalancear sigue en pie). *X solo gana re-estimando* (se cumple lo que el Council dijo que revertiría la decisión #5; decides tú).

Un ejemplo completo

La captura de arriba es la salida real sobre la cartera de 3 forzada del pool de prueba `Test_nikkei_ref` (raíz de prueba, capítulo 100; filtros relajados a 0,90). Son diez casi-clones de volatilidad parecida: los pesos de `risk_parity` salen 0,31 / 0,35 / 0,34, casi iguales, y ningún método gana. Sobre la cartera de 5 del mismo pool, lo mismo: la mayor diferencia es −0,007 con intervalo [−0,031, +0,019]. **Es una prueba de que el código funciona, no un hallazgo.**

Qué NO te dice

- **No elige el método.** Aunque uno gane, `weights.method` sigue en `equal` hasta que tú lo cambies.
- **No corrige por el número de comparaciones.** Con 6 brazos al 95 %, que alguno «gane» por azar puede pasar hasta una vez de cada siete aunque ninguno sea mejor de verdad. Un solo «sí» pide repetirlo en otro pool antes de creérselo.

- **Compara por Sharpe.** Un método puede bajar el drawdown sin subir el Sharpe; las otras cuatro columnas están para verlo, pero no deciden.
- **Con `oos1` corto hay poca potencia.** En 2 años de `oos1` solo una diferencia grande sale del ruido.
- **Con un solo tramo, re-estimar y no re-estimar son lo mismo**, y el comando lo avisa. Con el valor por defecto (3 años) y un `oos1` de 2-3 años pasa siempre.

Si algo falla

- `wf.confidence no tiene valor y no lleva defecto` — falta `--set wf.confidence=…`.
- `la cartera tiene un solo miembro` — no hay pesos que comparar.
- `No se pudo estimar <método>: sin volatilidad en los días de estimación` — una estrategia no se movió en la ventana de estimación (por ejemplo, con `wf.lookback_years` muy corto). Ese método se queda fuera; los demás siguen.
- `weights.method=…: solo «equal» puede elegirse` — sale en cualquier otro comando de cartera si intentas seleccionar un método con `--set`. Es a propósito.

104. Cartera · la incubación — el registro de la cartera elegida y la lectura de sus días nuevos

Qué pregunta responde

Una vez leído `oos2`, a la cartera no le queda histórico sin mirar. Lo único que la valida a partir de ahí son **los días que todavía no existían** cuando se eligió (Council, 2026-10-04; hito M9 de `portfolio/PLAN.md` §14.5). Este comando hace dos cosas:

1. **Registrar**: congela la cartera (quiénes, con qué pesos) y **cómo se la va a juzgar**, antes de ver un solo día nuevo.
2. **Leer**: dado un fichero con sus días nuevos, dice si ese tramo cae **dentro o fuera de la banda** de lo que la propia cartera hizo en tramos igual de largos de su historia.

> **LA INCUBACIÓN NO ESTÁ EN MARCHA.** Hoy nada produce esos días nuevos: ni el retest mensual en SQX de las barras posteriores a `oos2`, ni una cuenta real o demo conectada a este módulo. Y no hay ninguna cartera elegida de verdad. Lo que existe y está probado es el contrato (qué es una fila de incubación), el registro y la lectura. El comando lo dice en su primera línea cada vez.

Cuándo lo usas, y cuándo no

Cuando una cartera ha pasado su veredicto (capítulo 101 o 87) y vas a seguirla. **No** lo uses para decidir si una cartera es buena: eso es el veredicto. **No** registres una cartera «para probar» en la raíz real: un registro se escribe una vez y no se cambia.

Antes de empezar

- Una run con veredicto y su carpeta `handoff/` (capítulo 101).
- Tres decisiones tuyas, que **no tienen valor por defecto** y sin las que registrar se niega:

ajuste	qué decide	ejemplo
`incubation.figure`	qué se mide en el tramo nuevo: `pnl` (la ganancia total, en unidades de riesgo) o `sharpe` (ganancia media diaria entre lo que oscila)	`pnl`
`incubation.history`	con qué parte de la historia se compara	`[oos1, oos2]`
`incubation.band`	los dos percentiles que marcan la banda; fuera de ella, la lectura queda señalada. `[5, 100]` señala solo por abajo	`[5, 100]`

- Para leer, un fichero de filas con **exactamente** estas columnas (las mismas de `handoff/daily.parquet`):

columna	qué es
`day`	el día, posterior al último de la historia registrada
`identity`	la estrategia; solo las de la cartera
`pnl`	su resultado de ese día, en la misma unidad que el backtest archivado. **Una fila por estrategia y día de mercado, también los días a cero**
`segment`	siempre `incubation`
`source`	`paper` (retest en SQX de las barras nuevas) o `live` (una cuenta); uno solo por fichero

Cómo se ejecuta

```
python3 -m portfolio.common.construct.incubation.run register \
    --run ~/Desktop/AlgoData/portfolio/runs/2026-10-05T0954_Test_nikkei_ref_real \
    --set incubation.figure=pnl --set "incubation.history=[oos1, oos2]" --set "incubation.band=[5, 100]"

python3 -m portfolio.common.construct.incubation.run read --portfolio aca5 --rows dias_nuevos.parquet
```

flag	obligatorio	qué hace
`register --run <carpeta>`	sí	la run cuya cartera se congela
`register --set incubation.…`	sí, los tres	la comparación; queda escrita en el registro y ya no se cambia
`read --portfolio <huella>`	sí	el principio de la huella de la cartera registrada
`read --rows <fichero>`	no	el fichero de días nuevos (parquet o csv). Sin él, solo dice el estado: cuántas lecturas hay y la última

Tarda un segundo. No toca SQX.

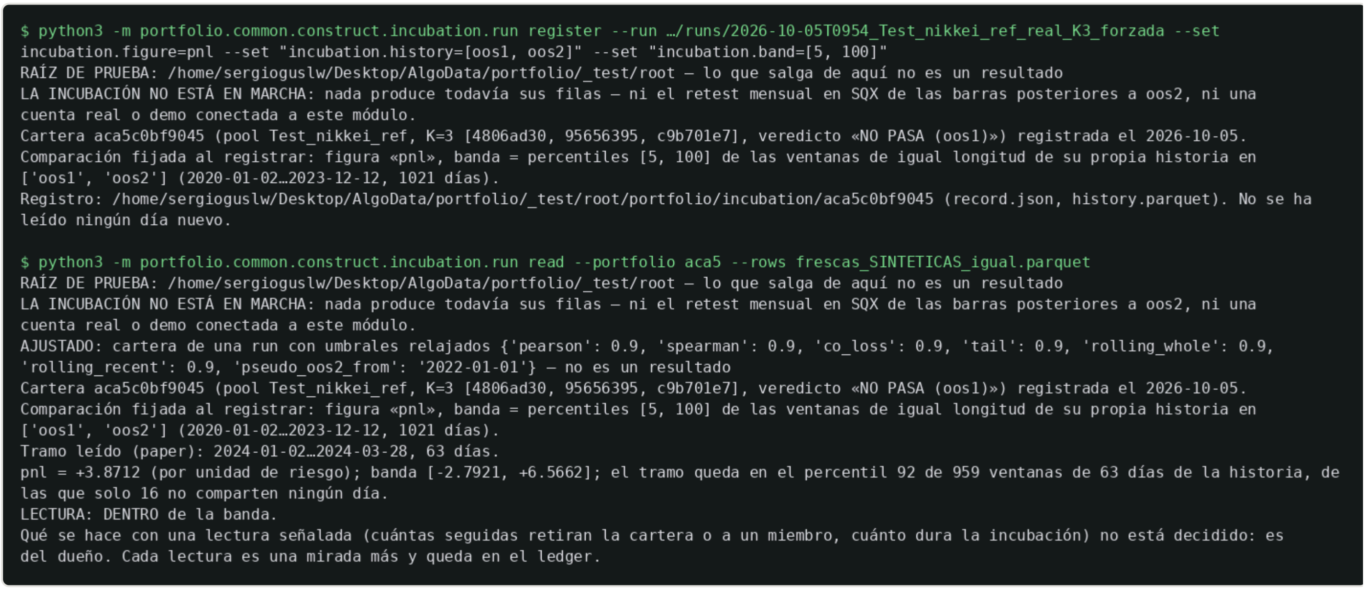
Qué produce

`AlgoData/portfolio/incubation/<huella>/`:

archivo	qué es
`record.json`	el registro: la cartera, sus pesos, la unidad de riesgo de cada estrategia, la comparación fijada, el tramo de historia. Se escribe una vez
`history.parquet`	los días de la historia contra los que se compara
`readings.jsonl`	una línea por lectura hecha

Y cada lectura añade **una fila al registro** (paso 27, tramo `incubation`): cada vez que miras cuenta como una mirada.

Cómo se lee el resultado



```
$ python3 -m portfolio.common.construct.incubation.run register --run …/runs/2026-10-05T0954_Test_nikkei_ref_real_K3_forzada --set
incubation.figure=pnl --set "incubation.history=[oos1, oos2]" --set "incubation.band=[5, 100]"
RAÍZ DE PRUEBA: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
LA INCUBACIÓN NO ESTÁ EN MARCHA: nada produce todavía sus filas — ni el retest mensual en SQX de las barras posteriores a oos2, ni una
cuenta real o demo conectada a este módulo.
Cartera aca5c0bf9045 (pool Test_nikkei_ref, K=3 [4806ad30, 95656395, c9b701e7], veredicto «NO PASA (oos1)») registrada el 2026-10-05.
Comparación fijada al registrar: figura «pnl», banda = percentiles [5, 100] de las ventanas de igual longitud de su propia historia en
['oos1', 'oos2'] (2020-01-02…2023-12-12, 1021 días).
Registro: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root/portfolio/incubation/aca5c0bf9045 (record.json, history.parquet). No se ha
leído ningún día nuevo.

$ python3 -m portfolio.common.construct.incubation.run read --portfolio aca5 --rows frescas_SINTETICAS_igual.parquet
RAÍZ DE PRUEBA: /home/sergioguslw/Desktop/AlgoData/portfolio/_test/root — lo que salga de aquí no es un resultado
LA INCUBACIÓN NO ESTÁ EN MARCHA: nada produce todavía sus filas — ni el retest mensual en SQX de las barras posteriores a oos2, ni una
cuenta real o demo conectada a este módulo.
AJUSTADO: cartera de una run con umbrales relajados {'pearson': 0.9, 'spearman': 0.9, 'co_loss': 0.9, 'tail': 0.9, 'rolling_whole': 0.9,
'rolling_recent': 0.9, 'pseudo_oos2_from': '2022-01-01'} — no es un resultado
Cartera aca5c0bf9045 (pool Test_nikkei_ref, K=3 [4806ad30, 95656395, c9b701e7], veredicto «NO PASA (oos1)») registrada el 2026-10-05.
Comparación fijada al registrar: figura «pnl», banda = percentiles [5, 100] de las ventanas de igual longitud de su propia historia en
['oos1', 'oos2'] (2020-01-02…2023-12-12, 1021 días).
Tramo leído (paper): 2024-01-02…2024-03-28, 63 días.
pnl = +3.8712 (por unidad de riesgo); banda [-2.7921, +6.5662]; el tramo queda en el percentil 92 de 959 ventanas de 63 días de la historia, de
las que solo 16 no comparten ningún día.
LECTURA: DENTRO de la banda.
Qué se hace con una lectura señalada (cuántas seguidas retiran la cartera o a un miembro, cuánto dura la incubación) no está decidido: es
del dueño. Cada lectura es una mirada más y queda en el ledger.
```

- **«Comparación fijada al registrar»**: lo que tú decidiste antes de ver nada. La lectura usa eso y no lo que haya ahora en la configuración.
- **«pnl = +3,87 (por unidad de riesgo)»**: lo que ganó la cartera en el tramo nuevo. Una unidad de riesgo es lo que se arriesga por operación; +3,87 son casi cuatro de esas.
- **«banda [−2,79, +6,57]»**: entre qué valores quedó la cartera en el 95 % de los tramos de 63 días de su propia historia (del percentil 5 al 100, lo que tú fijaste).
- **«percentil 92 de 959 ventanas … de las que solo 16 no comparten ningún día»**: el tramo nuevo fue mejor que el 92 % de los tramos históricos de la misma longitud. Ojo al segundo número: las 959 ventanas se pisan unas a otras; **independientes hay 16**, y esa es la muestra real de la banda.
- **«LECTURA: DENTRO de la banda»** o **«POR DEBAJO de la banda — señalada»**.

Un ejemplo completo

La captura es la salida real en la raíz de prueba (capítulo 100): se registra la cartera de 3 forzada del pool `Test_nikkei_ref` y se le lee un fichero de 63 días **sintéticos** (días de su propia historia barajados, hechos para probar el comando: no son días nuevos de verdad). Queda dentro de la banda, como debe.

Qué NO te dice

- **No dice qué hacer con una lectura señalada.** Cuántas seguidas retiran la cartera, o a un miembro, y cuánto dura la incubación, no está decidido.
- **La banda no es exacta.** Con diez años de historia y tramos de un año hay solo diez tramos independientes: en la prueba, una cartera sana quedó por debajo del «percentil 5» casi un 10 % de las veces, no un 5 %.
- **Mirar cada mes multiplica las falsas alarmas.** En la misma prueba, con 12 lecturas mensuales una cartera sana fue señalada **al menos una vez en una de cada cuatro** simulaciones. Una señal suelta no es una cartera rota.
- **Una cartera que ha perdido su ventaja tarda en verse.** Con una ventaja como la de la prueba (Sharpe diario 0,2, muy alta), la lectura de los 12 meses la señaló el 94 % de las veces. Con una ventaja normal hará falta bastante más tiempo.

- **No convierte una cuenta real en filas.** Pasar las operaciones de una cuenta (capítulo 89) a estas filas, al tamaño del backtest, no está construido.

Si algo falla

- `la comparación no tiene valor y no lleva defecto` — faltan los `--set incubation.…`.
- `la cartera … ya está registrada` — un registro se escribe una vez.
- `las filas no cumplen el contrato de incubación` — debajo lista cada problema: una estrategia que no es de la cartera, un día que no es posterior a la historia, un `segment` o `source` que no toca.